Java

1, Eight Basic Data Type (Primitive Type)

byte, short, int, long, float, double, char, boolean
8bits 16, 32, 64, 32, 64, 16, 1

Wrapper
Byte, Short, Integer, Long, Float, Double, Character, Boolean

autoboxing
unboxing

2. String/ StringBuilder/ StringBuffer

String is immutable
StringBuilder is mutable, not thread safe
StringBuffer is mutable, thread safe

Java is Pass By Value

```Java
public class Solution {

    public static void main(String[] args) {
        String s1 = "Java";
        concat(s1);
        System.out.println(s1);  // Java

        StringBuilder s2 = new StringBuilder("java");
        concat1(s2);
//        System.out.println(s2);

        StringBuffer s3 = new StringBuffer("java");
        concat2(s3);
//        System.out.println(s3);
    }
```

```
    public static void concat(String ref) {
        ref = ref + "Python";
        System.out.println(ref); // JavaPython
    }

    public static void concat1(StringBuilder s2) {
        s2.append("builder");
    }

    public static void concat2(StringBuffer s3) {
        s3.append("buffer");
    }

}
```

3. equals() / hashCode()

in Java, all the classes are inherited from **Object** class

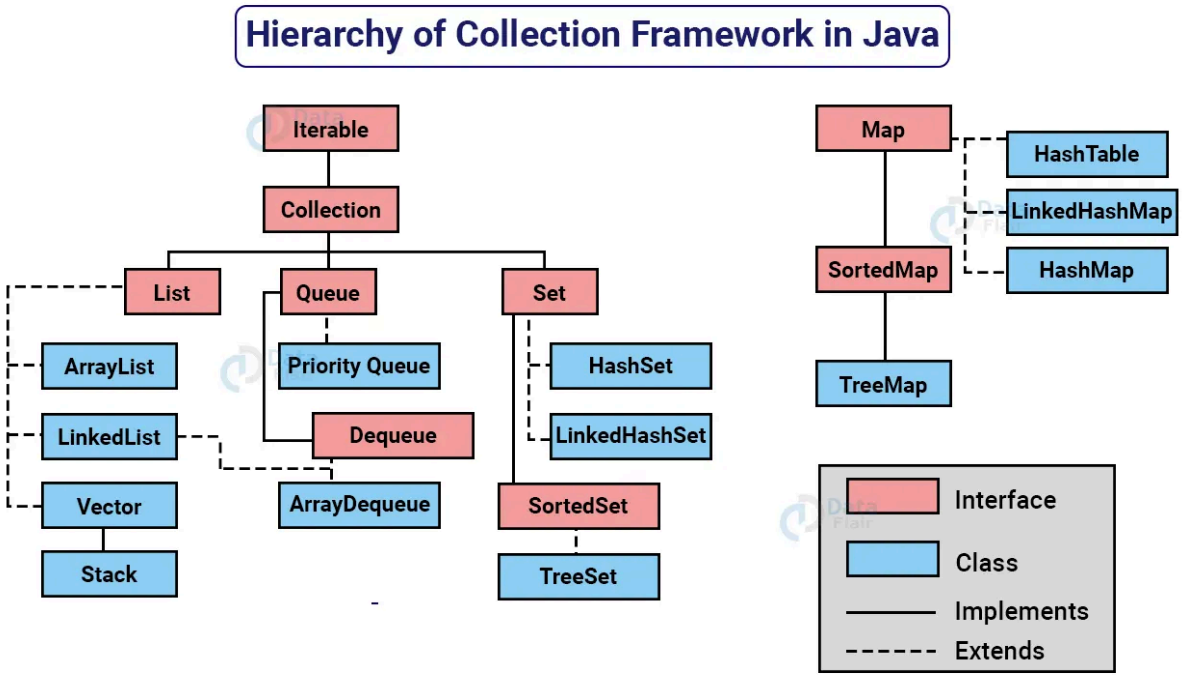
equals() vs ==

Java

```java
package org.example;

import java.util.Objects;

public class Solution {

    public static void main(String[] args) {
        Student A = new Student("A", 20);
        Student A1 = new Student("A", 20);

        System.out.println(A.equals(A1)); // true

    }

}
```

```java
class Student {
    String name;
    int age;

    public Student(String name, int age) {
        this.name = name;
        this.age = age;
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof Student student)) return false;
        return age == student.age && Objects.equals(name, student.name);
    }

    @Override
    public int hashCode() {
        return Objects.hash(name, age);
//        return Objects.hash(name) + Objects.hash(age);
    }
}
```

4. Data Structures



Collection vs Collections
Collection interface
Collections class

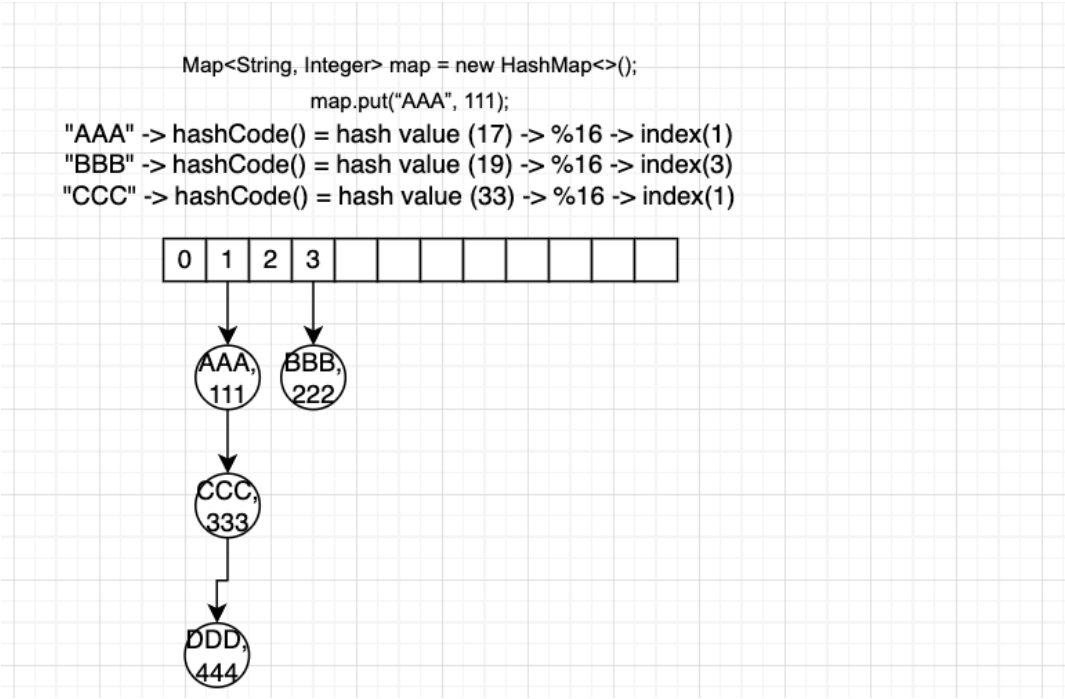
<u>ArrayList vs LinkedList</u>

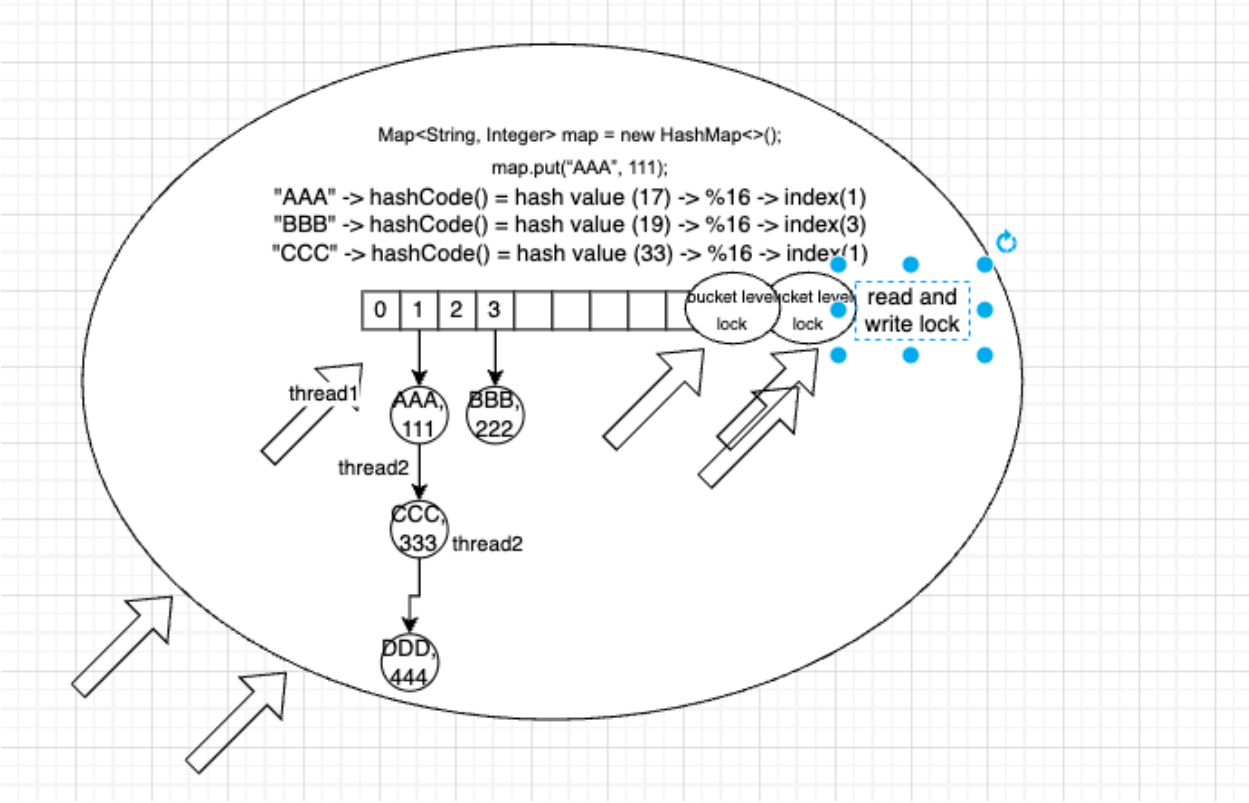
HashMap
Map<String, Integer> map = new HashMap<>();
map.put(“AAA”, 111);
map.put(“BBB”, 222);
map.put(“CCC”, 333);

Hash Collision

HashMap vs HashTable vs ConcurrentHashMap

- HashMap: not thread safe
- HashTable: thread safe, Object level
- ConcurrentHashMap, thread safe, bucket level





Array vs ArrayList vs LinkedList
HashMap, LinkedHashMap, HashTable, ConcurrentHashMap, TreeMap
HashSet, LinkedHashSet, TreeSet

PriorityQueue (MinHeap, MaxHeap)
Queue, Stack
Deque (ArrayDeque)

5. Comparator vs Comarable

Both are interface
Comparator -> compare();
Comparable -> compareTo();

Java

```java
package org.example;

import java.util.*;

public class Solution {

    public static void main(String[] args) {

        List<Student> stuList = new ArrayList<>();
        stuList.add(new Student("A", 10));
        stuList.add(new Student("B", 20));
//        Collections.sort(stuList, new MyComparator());
//        System.out.println(stuList);

//        Collections.sort(stuList, (Student s1, Student s2) -> {
//            return s2.name.compareTo(s1.name);
//        });
//        Collections.sort(stuList, (s1,  s2) -> s2.name.compareTo(s1.name));

        Collections.sort(stuList);
        System.out.println(stuList);



    }

}

class Student implements Comparable<Student>{
    String name;
    int age;
```

```
    public Student(String name, int age) {
        this.name = name;
        this.age = age;
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof Student student)) return false;
        return age == student.age && Objects.equals(name, student.name);
    }

    @Override
    public int hashCode() {
        return Objects.hash(name, age);
//        return Objects.hash(name) + Objects.hash(age);
    }

    @Override
    public String toString() {
        return "Student{" +
                "name='" + name + '\'' +
                ", age=" + age +
                '}';
    }

    @Override
    public int compareTo(Student o) {
        if (this.age == o.age) {
            return 0;
        }
        return this.age > o.age? 1: -1;
    }
}

class MyComparator implements Comparator<Student> {

    @Override
    public int compare(Student o1, Student o2) {
        return o2.name.compareTo(o1.name);
    }
}
```

6. JVM Architecture

JVM vs JRE vs JDK

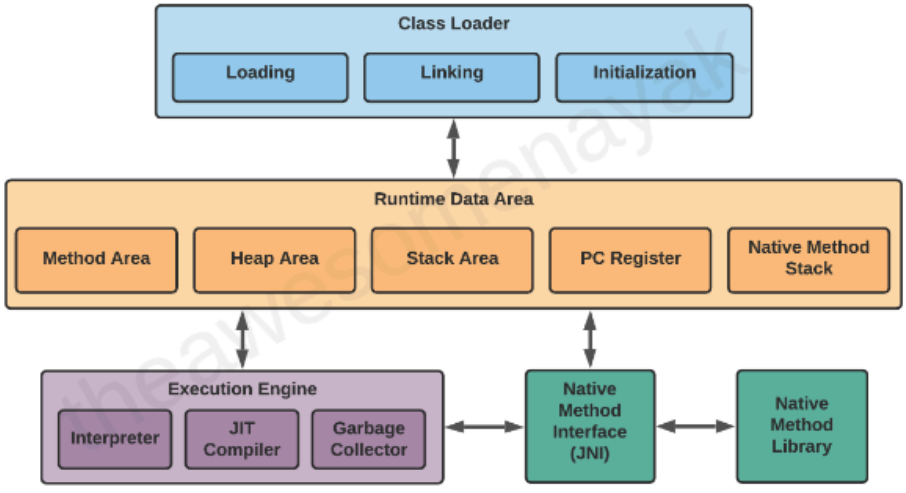
```
JDK
├── JRE
│   ├── JVM
│   └── Core Libraries
└── Dev Tools (javac, jar, javadoc...)
```

Solution.java →(javac)-> Solution.class

Compiled language vs Interpreted language vs Java
Java: compile once, run everywhere

JVM Architecture

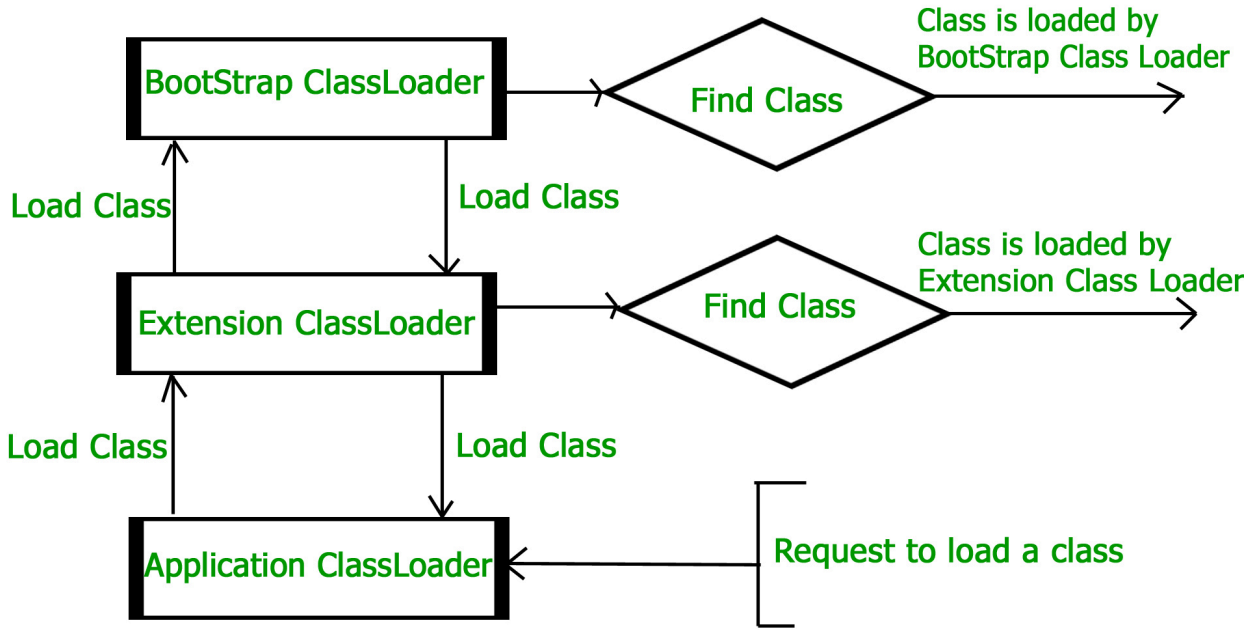
1. Class Loader
2. Runtime Memory/Data Area
3. Execution Engine



Bootstrap Class Loader - This is the root class loader. It is the superclass of Extension Class Loader and loads the standard Java packages like **java.lang, java.net, java.util, java.io, and**

so on. These packages are present inside the rt.jar file and other core libraries present in the $JAVA_HOME/jre/lib directory.
Extension Class Loader - This is the subclass of the Bootstrap Class Loader and the superclass of the Application Class Loader. This loads the extensions of standard Java libraries which are present in the **$JAVA_HOME/jre/lib/ext directory**.
Application Class Loader - This is the final class loader and the subclass of Extension Class Loader. It loads the files present on the classpath. By default, the classpath is set to the current directory of the application. The classpath can also be modified by adding the -classpath or -cp command line option.

https://www.geeksforgeeks.org/java/classloader-in-java/



Method Area

All the class level data such as the run-time constant pool, field, and method data, and the code for methods and constructors, are stored here.

If the memory available in the method area is not sufficient for the program startup, the JVM throws an `OutOfMemoryError`.

For example, assume that you have the following class definition:

```
public class Employee {

  private String name;
  private int age;

  public Employee(String name, int age) {

    this.name = name;
    this.age = age;
  }
```

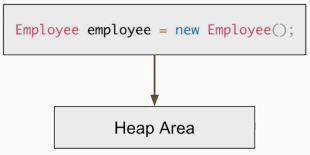
Constant Pool: Integer Intern Pool, String Intern Pool
int num = 100; 1000// what will happen?
int num2 = 100; 1000//
num == num2;//
Integer num3 = new Integer(100);

Heap Area

All the objects and their corresponding instance variables and arrays are stored here.

This is the run-time data area from which memory for all class instances and arrays is allocated.

The heap is created on the virtual machine start-up, and there is only one heap area per JVM.



Is Stack Thread safe? yes

2. Heap Area
All the objects, their related instance variables, and arrays are stored in the heap.

Let's take the following code sample as an example.

Student student = new Student();
In the above code example, an instance of Student is created and loaded into the heap area.

3. Stack Area
Java language stacks store local variables and their partial results. Each thread has its own JVM stack, created simultaneously as the thread is created.

4. PC Registers
The PC register stores the address of the Java virtual machine instruction that is currently executing. In Java, each thread has its own PC register.

Native Method Stack
Native method stacks hold the instructions of native code, which depends on the native library. It is written in another language instead of Java. For every new thread, a separate native method stack is also allocated.

Execution Engine
The Execution Engine runs the application by executing the code present in each class.

7. Garbage Collection

How: Mark, Sweep, Compact
Types: Serial GC, Parallel GC, G1 GC
CMS (Concurrent Mark Sweep GC) deprecated since Java 9, completely removed in java 14

Young generation, old generation, permanent generation (removed, replaced by metaspace)

OOM ->

java.lang.OutOfMemoryError: Java heap space

java.lang.OutOfMemoryError: GC overhead limit exceeded

java.lang.OutOfMemoryError: Direct buffer memory

java.lang.OutOfMemoryError: unable to create new native thread

java.lang.OutOfMemoryError: Requested array size exceeds VM limit

java.lang.OutOfMemoryError: Metaspace

8. Keywords

reserved keywords: 53

- Reserved words for data types:
  - byte,short,int,long,float,double,char,boolean
- Reserved words for flow control:
  - if ,else,switch ,case ,default ,for ,do ,while ,break ,continue,return
- Keywords for modifiers:
  - public ,private ,protected,static,final ,abstract ,synchronized, transient,volatile
- keywords for exception handling:
  - try,catch,finally ,throw,throws
- Class related keywords:
  - class,package,import,extends,implements,interface
- Object related keywords:
  - new,instanceof ,super ,this,

Friday:
Mock Interview
Focus -> Feedback -> Fix

final

- final variable (need initialize, can't re-reference)

```java
no usages
public class Solution {

    no usages
    final int a = 0;

    no usages
    public static void main(String[] args) {
        final String str = "abc";
        str = "bcd";

    }

}
```

-

```java
class People {
    no usages
    final public String getName() {
        return "name";
    }
}

no usages
class Student extends People {
    no usages
    @Override
    public String getName() {

    }
}
```

- final class (can’t be inherited)

immutable class

- final class
- private final fields
- no setter
- return unmodified collection or deep copy of the collection

```java
package org.example;

import java.util.*;
//import java.lang.*;

public class Solution {



    public static void main(String[] args) {
        int id = 1;
        String name = "Bob";
        List<Integer> list = List.of(1, 2, 3);
       // List<Integer> list = Collections.unmodifiableList(Arrays.asList(1,2,3));
        MyImmutableClass obj = new MyImmutableClass(id, name, list);


        List<Integer> list2 = obj.getList();
        list2.add(4);
```

```
        System.out.println(obj.getList());


    }

}

final class MyImmutableClass {
    private final int id;
    private final String name;
    private final List<Integer> list;  // unmodified list or return deep copy


    public MyImmutableClass(int id, String name, List<Integer> list) {
        this.id = id;
        this.name = name;
        this.list = list;
    }

    public int getId(){
        return id;
    }

    public String getName(){
        return name;
    }

    public List<Integer> getList() {
        List<Integer> res = new ArrayList<>(list);
        return res;
    }
}
```

final, finally, finalize

Static

- block
- variable
- method
- class

Map<String, Integer> map = new HashMap<>();
for (Map.Entry<String, Integer> entry: map.entrySet()) {

}

Java

```java
package org.example;

import java.util.*;
//import java.lang.*;

public class Solution {

    static int a;
    static int b;

    static {
        a = 0;
        System.out.println("initializing");
    }

    public static void main(String[] args) {
        Car car1 = new Car(10);

        Car car2 = new Car(20);

        System.out.println(Car.brand);
        System.out.println(Car.getBrand());
    }
}

class Car {
    static String brand = "ford";

    int price;

    public static String getBrand() {
        return brand;
    }

    public Car(int price) {
        this.price =price;
    }


    static class Engine {
```

```
    }

}
```

volatile

- read from the main memory, guarantee the visibility of each thread
- prevent instruction reordering

synchronized: lock

transient: serialization

implements vs extends
class extends class
class implements interface
interface extends interface

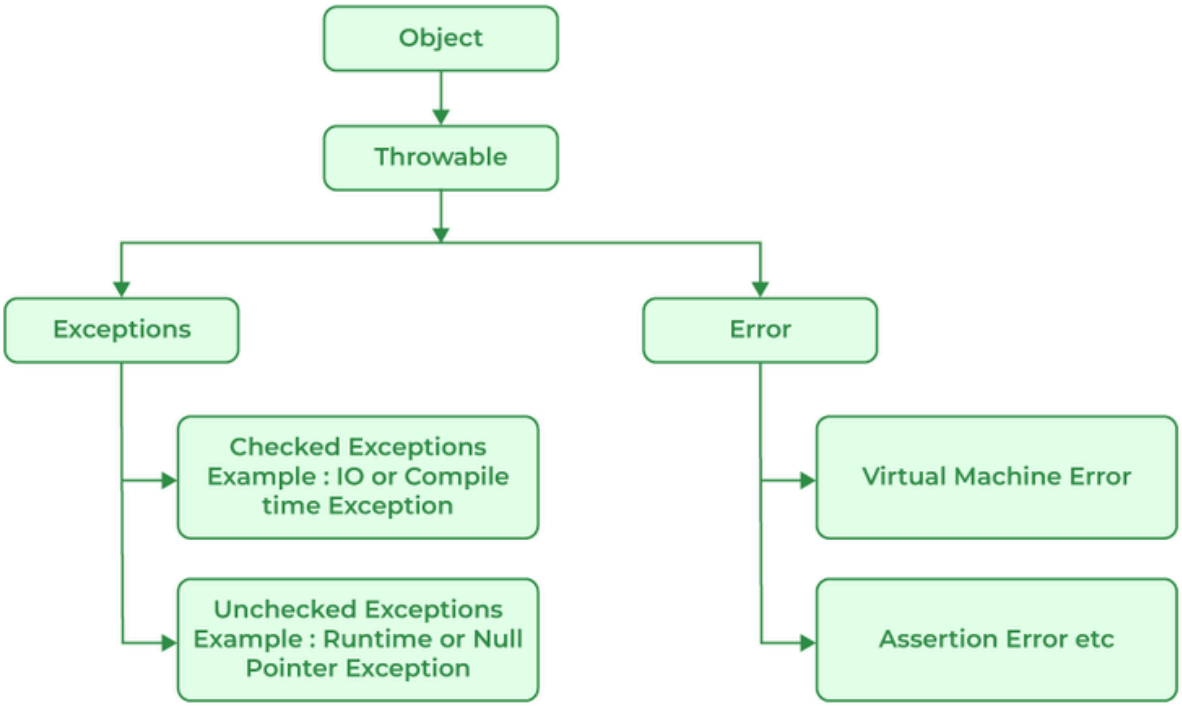
9. OOP

- Abstraction
    - abstract class
    - Interface
- Encapsulation
    - private variable
    - getter/ setter
- Inheritance
    - extends
    - implements
- Polymorphism
    - override
    - overload

```Java
    public int getPrice() {
        return 0;
    }
```

```
    public int getPrice(String model) {
        return  0;
    }
```

10. Exception



checked exception(compile time): we need to handle

- ClassNotFound, IOExeception, SQLException

unchecked exception(runtime exception): we are not required to handle

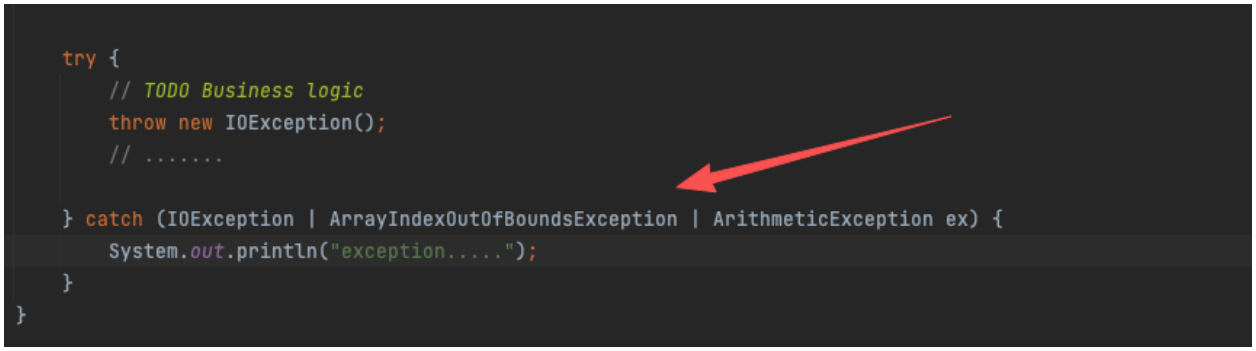
- ArithmeticException, ArrayIndexOutOfBoundsException, ClassCastException…

```
Java
   public static void main(String[] args) {
//        System.out.println(8/0);
```

```
        int[] arr = new int[3];
        System.out.println(arr[100]);

    }
```

how to handle checked exception

- try catch
- throws



try with resources
AutoCloseable -> close()

try {

} catch () {

} finally {
        connection.close();
}

11. Generics

Java

```java
package org.example;

import java.io.IOException;
import java.util.*;
//import java.lang.*;

public class Solution {
    public static void main(String[] args) {
//        Node node = new Node();
        Node<Integer, Integer> node = new Node<>();
        Node<String, Double> node2 = new Node<>();
    }

    public static <E> E getElement(E[] arr) {
        return arr[0];
    }

}

class Node <K, V> {
    K key;
    V value;
}

class Node3 {
    String key;
    Integer value;
}

class Node1 {
    Integer key;
    Double value;
}
```

```
class Node2 {
    String key;
    Float value;
}
```

12. Java 8 Features

java 8/11/17/21

Lambda
(arguments list) -> {body}

int method(Integer num, List<String> strs){
        // business logic
        return 0;
}

Functional Interface
- one abstract method

```Java
package org.example;

import java.io.IOException;
import java.util.*;
//import java.lang.*;

public class Solution {
    public static void main(String[] args) {
        Queue<Integer> maxHeap = new PriorityQueue<>((Integer e1, Integer e2) ->
{return e2 - e1;});
        Queue<Integer> maxHeap2 = new PriorityQueue<>((Integer e1, Integer e2) ->
e2 - e1 );
        Queue<Integer> maxHeap3 = new PriorityQueue<>((e1,  e2) ->  e2 - e1 );
```

```
        Paper obj = new Paper();
        obj.draw();

        Drawable obj2 = () -> {
            System.out.println("draw a square");
        };
        obj2.draw();

    }


}

@FunctionalInterface // optional
interface Drawable {
    public void draw();
}

class Paper implements Drawable {
    @Override
    public void draw() {
        System.out.println("draw a circle");
    }
}
```

Java predefined - functional interface
Predicate
public <T> boolean test(T t);
Function
public <R, T> R apply(T t);
Consumer
pubic <T> void accept(T t);
Supplier
public <R> R get();

Comparator
Runnable
Callable
BiPredicate
BiFunction
BiConsumer

….

```
Supplier<Double> generateRandomNum = () -> Math.random();
System.out.println(generateRandomNum.get());
```

https://docs.oracle.com/javase/8/docs/api/java/util/function/package-summary.html

https://www.geeksforgeeks.org/java/java-functional-interfaces/

Functional Interface

- only one abstract method
- any number of default and static methods

Java

```
@FunctionalInterface
interface StringProcessor {
    String process(String input);

    static boolean isEmpty(String input) {
        return input == null || input.isEmpty();
    }

    static boolean isEmpty2(String input) {
        return input == null || input.isEmpty();
    }

    default String toUpperCase(String input) {
        return input.toUpperCase();
    }

    default String toLowerCase(String input) {
        return input.toLowerCase();
    }

}
```

Optional

- deal with NullPointerException

```java
package org.example;

import java.io.IOException;
import java.util.*;
import java.util.function.*;
//import java.lang.*;

public class Solution {
    public static void main(String[] args) {
       String[] arr = new String[3];

//        System.out.println(arr[0].length());
//       if (arr[0] == null) {
//           System.out.println("nothing here");
//       } else {
//           System.out.println(arr[0].length());
//       }

       Optional opt = Optional.ofNullable(arr[0]);
        System.out.println(opt.orElse("nothing here"));



    }


}
```

https://docs.oracle.com/javase/8/docs/api/java/util/Optional.html

Stream API (lazy computation)

- intermediate operations: return a stream
- terminal operations: return non stream, like collection, object, return void

https://www.baeldung.com/java-8-streams

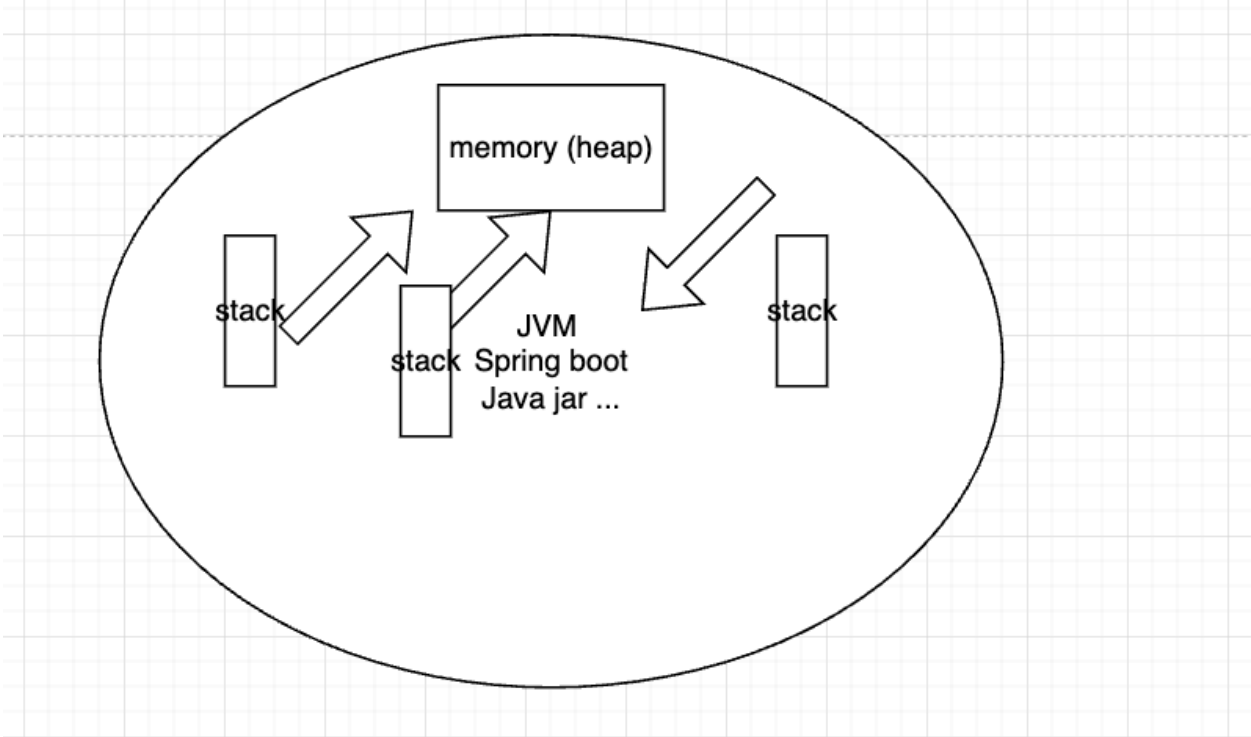
```java
List<Integer> list = new ArrayList<>(Arrays.asList(1, 2, 3, 4, 5));
List<Integer> res = list.stream().filter(e -> e > 2).map(e ->
e*2).collect(Collectors.toList());
System.out.println(res);
```

Method Reference
CompleteableFuture

Default keyword vs Java default scope

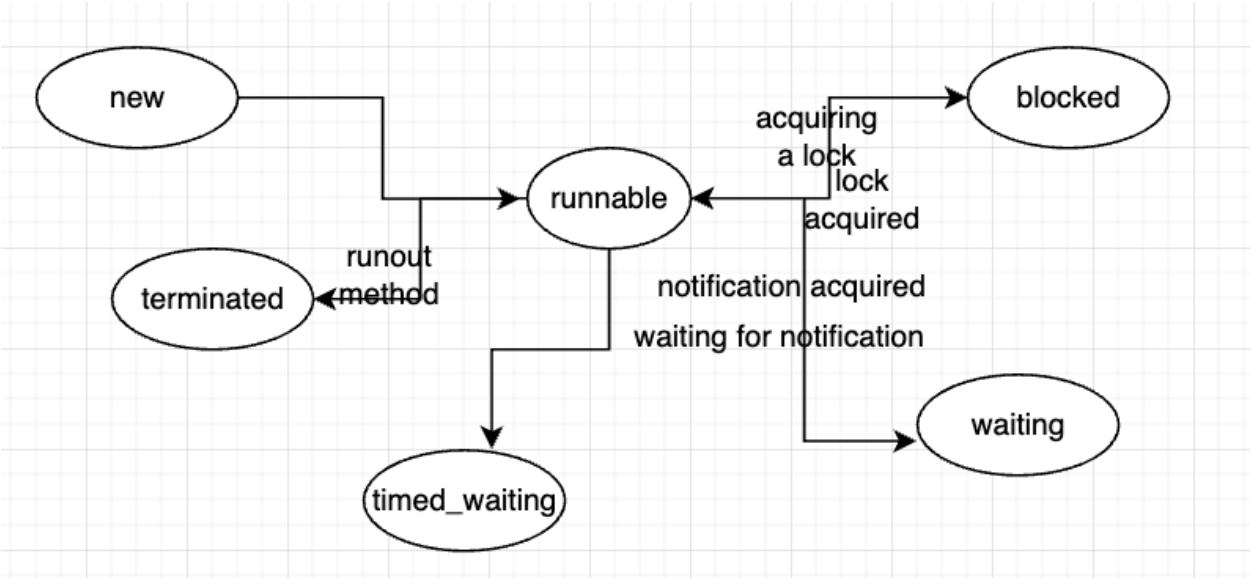
13. Multithreading

Thread vs Process



Thread state / Lifecycle

- **new**
- **runnable**
- **blocked**
- **waiting**
- **timed_waiting**
- **terminated**



https://www.geeksforgeeks.org/java/lifecycle-and-states-of-a-thread-in-java/

Four ways to create thread

- **extends Thread**
- **implements Runnable Interface**
  - **return void**
  - **can't handle exception**
- **implements Callable Interface**
  - **return Object**
  - **can handle exception**
- **thread pool. (ExecutorService)**

```java
package org.example;

import java.io.IOException;
import java.util.*;
import java.util.concurrent.Callable;
import java.util.concurrent.ExecutionException;
import java.util.concurrent.FutureTask;
import java.util.function.*;
import java.util.stream.Collectors;
//import java.lang.*;

public class Solution {
```

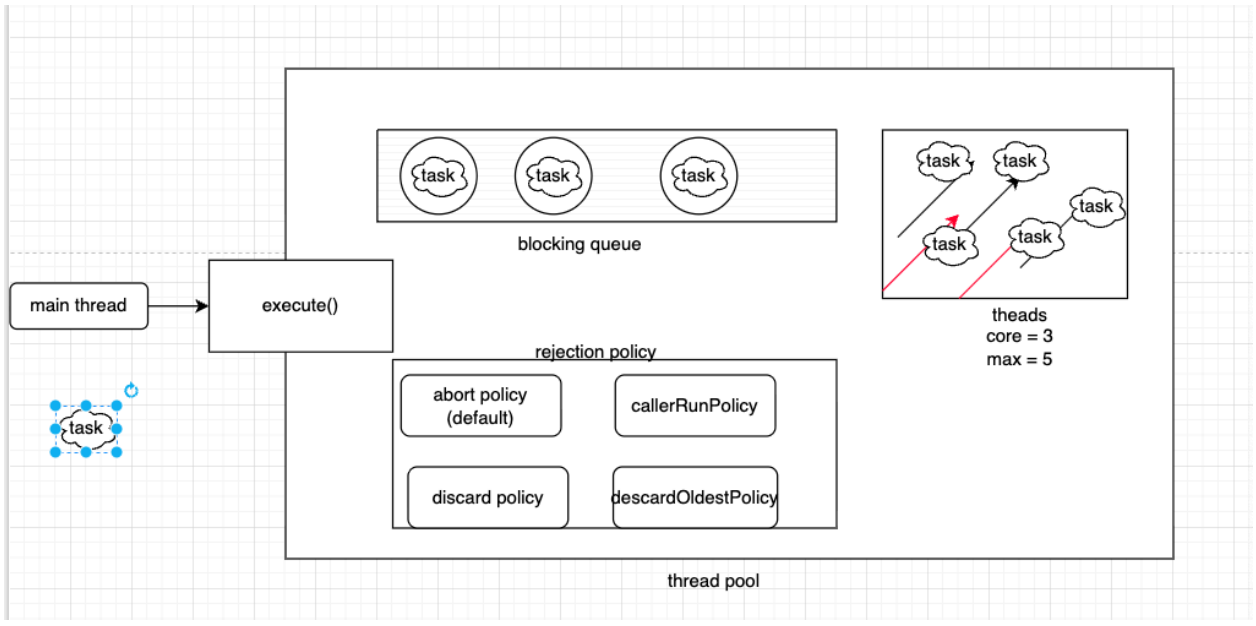
```
    public static void main(String[] args) throws ExecutionException,
InterruptedException {

        System.out.println("main thread " + Thread.currentThread().getName());

        Thread t1 = new MyThread();
        t1.start();

        Thread t2 = new Thread(new RunnableThread());
        t2.start();

        FutureTask task = new FutureTask(new CallableThread());
        Thread t3 = new Thread(task);
        t3.start();
        System.out.println(task.get());

        System.out.println("main thread " + Thread.currentThread().getName());

    }

}

class MyThread extends Thread {
    @Override
    public void run() {
        System.out.println("extends thread class " +
Thread.currentThread().getName());
    }
}

class RunnableThread implements Runnable {
    @Override
    public void run() {
        System.out.println("implements Runnable Interface " +
Thread.currentThread().getName());
    }
}

class CallableThread implements Callable {

    @Override
    public Integer call() throws Exception {
        System.out.println("implements Callable Interface " +
Thread.currentThread().getName());
        return 200; // http status code
```

```
    }
}
```

ThreadPool

```
/**
* Creates a new {@code ThreadPoolExecutor} with the given initial
* parameters, the
* {@linkplain Executors#defaultThreadFactory default thread factory}
* and the {@linkplain ThreadPoolExecutor.AbortPolicy
* default rejected execution handler}.
*
* <p>It may be more convenient to use one of the {@link Executors}
* factory methods instead of this general purpose constructor.
*
* @param corePoolSize the number of threads to keep in the pool, even
*        if they are idle, unless {@code allowCoreThreadTimeOut} is set
* @param maximumPoolSize the maximum number of threads to allow in the
*        pool
* @param keepAliveTime when the number of threads is greater than
*        the core, this is the maximum time that excess idle threads
*        will wait for new tasks before terminating.
* @param unit the time unit for the {@code keepAliveTime} argument
* @param workQueue the queue to use for holding tasks before they are
*        executed.  This queue will hold only the {@code Runable}
*        tasks submitted by the {@code execute} method.
* @throws IllegalArgumentException if one of the following holds:<br>
*         {@code corePoolSize < 0}<br>
*         {@code keepAliveTime < 0}<br>
*         {@code maximumPoolSize <= 0}<br>
*         {@code maximumPoolSize < corePoolSize}
* @throws NullPointerException if {@code workQueue} is null
*/
public ThreadPoolExecutor(int corePoolSize,
                          int maximumPoolSize,
                          long keepAliveTime,
                          TimeUnit unit,
                          BlockingQueue<Runnable> workQueue) {
   this(corePoolSize, maximumPoolSize, keepAliveTime, unit, workQueue,
        Executors.defaultThreadFactory(), defaultHandler);
```

```
Creates a new ThreadPoolExecutor with the given initial parameters.

Params: corePoolSize – the number of threads to keep in the pool, even if they are idle, unless
        allowCoreThreadTimeOut is set
        maximumPoolSize – the maximum number of threads to allow in the pool
        keepAliveTime – when the number of threads is greater than the core, this is the maximum
        time that excess idle threads will wait for new tasks before terminating.
        unit – the time unit for the keepAliveTime argument
        workQueue – the queue to use for holding tasks before they are executed. This queue will
        hold only the Runnable tasks submitted by the execute method.
        threadFactory – the factory to use when the executor creates a new thread
        handler – the handler to use when execution is blocked because the thread bounds and
        queue capacities are reached
Throws: IllegalArgumentException – if one of the following holds:
        corePoolSize < 0
        keepAliveTime < 0
        maximumPoolSize <= 0
        maximumPoolSize < corePoolSize
        NullPointerException – if workQueue or threadFactory or handler is null
public ThreadPoolExecutor( @Range(from = 0, to = java.lang.Integer.MAX_VALUE) int corePoolSize,
                           @Range(from = 1, to = java.lang.Integer.MAX_VALUE) int maximumPoolSize,
                           @Range(from = 0, to = java.lang.Long.MAX_VALUE) long keepAliveTime,
                           @NotNull TimeUnit unit,
                           @NotNull BlockingQueue<Runnable> workQueue,
                           @NotNull ThreadFactory threadFactory,
                           @NotNull RejectedExecutionHandler handler) {
    if (corePoolSize < 0 ||
        maximumPoolSize <= 0 ||
        maximumPoolSize < corePoolSize ||
        keepAliveTime < 0)
        throw new IllegalArgumentException();
    if (workQueue == null || threadFactory == null || handler == null)
```



```java
package org.example;

import java.io.IOException;
import java.util.*;
import java.util.concurrent.*;
import java.util.function.*;
import java.util.stream.Collectors;
//import java.lang.*;

public class Solution {
    public static void main(String[] args) throws ExecutionException,
InterruptedException {

        ExecutorService threadPool = new ThreadPoolExecutor(
                3,
                5,
                2L,
                TimeUnit.SECONDS,
                new ArrayBlockingQueue<>(3),
                Executors.defaultThreadFactory(),
                new ThreadPoolExecutor.AbortPolicy()
        );

        for (int i=1; i<=9; i++) {
            threadPool.execute(() -> {
                System.out.println(Thread.currentThread().getName() + " is
working");
            });
        }

    }

}
```

predefined thread pool

```
ExecutorService tp1 = Executors.newSingleThreadExecutor();
ExecutorService tp2 = Executors.newFixedThreadPool(3);
ExecutorService tp3 = Executors.newCachedThreadPool();
```

→ OutOfMemoryError

CompletableFuture + ThreadPool
runAsync/ supplyAsync

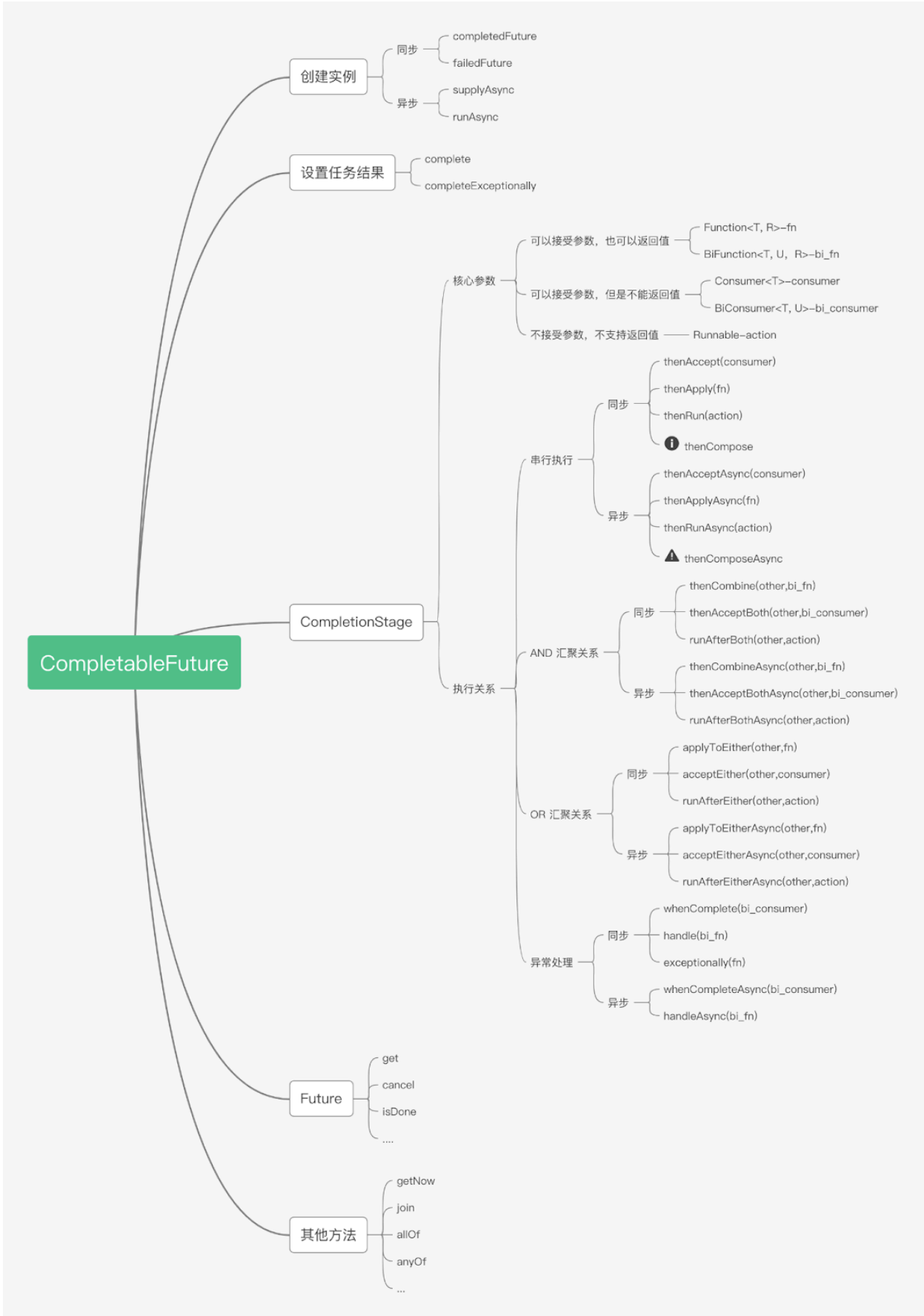
theApply, theApplyAsync
handle, exceptionally
thenCompose, thenCombine
allOf, anyOf
etc…..

Java

```java
package org.example;

import java.io.IOException;
import java.util.*;
import java.util.concurrent.*;
import java.util.function.*;
import java.util.stream.Collectors;
//import java.lang.*;

public class Solution {
    public static void main(String[] args) throws ExecutionException,
InterruptedException {

        System.out.println("main thread starts");

        CompletableFuture<Void> future = CompletableFuture.runAsync(()-> {

            try {
                System.out.println("thread 1 starts working");
                Thread.sleep(5000);
                System.out.println("thread 1 finished the task");
            } catch (InterruptedException e) {
                throw new RuntimeException(e);
            }

        });

        future.get();
        System.out.println("main thread is done");

        CompletableFuture<String> future1 = CompletableFuture.supplyAsync(()-> {

            try {
                System.out.println("thread 1 starts working");
                Thread.sleep(5000);
//                System.out.println("thread 1 finished the task");
            } catch (InterruptedException e) {
                throw new RuntimeException(e);
```

```
            }
            return "thread 1 finished the task";

        });

    }

}
```

```
api() {
        // will call 10 external api
        // how to mange those api call

        List<CompletableFuture<String>> calls = new ArrayList<>(...);
}
```


CompletableFuture
创建实例
同步
completedFuture
failedFuture
异步
supplyAsync
runAsync
设置任务结果
complete
completeExceptionally
CompletionStage
核心参数
可以接受参数，也可以返回值
Function<T, R>-fn
BiFunction<T, U，R>-bi_fn
可以接受参数，但是不能返回值
Consumer<T>-consumer
BiConsumer<T, U>-bi_consumer
不接受参数，不支持返回值
Runnable-action
执行关系
串行执行
同步
thenAccept(consumer)
thenApply(fn)
thenRun(action)
thenCompose
异步
thenAcceptAsync(consumer)
thenApplyAsync(fn)
thenRunAsync(action)
thenComposeAsync
AND 汇聚关系
同步
thenCombine(other,bi_fn)
thenAcceptBoth(other,bi_consumer)
runAfterBoth(other,action)
异步
thenCombineAsync(other,bi_fn)
thenAcceptBothAsync(other,bi_consumer)
runAfterBothAsync(other,action)
OR 汇聚关系
同步
applyToEither(other,fn)
acceptEither(other,consumer)
runAfterEither(other,action)
异步
applyToEitherAsync(other,fn)
acceptEitherAsync(other,consumer)
runAfterEitherAsync(other,action)
异常处理
同步
whenComplete(bi_consumer)
handle(bi_fn)
exceptionally(fn)
异步
whenCompleteAsync(bi_consumer)
handleAsync(bi_fn)
Future
get
cancel
isDone
....
其他方法
getNow
join
allOf
anyOf
...

14. Lock

synchronized

- class
- code block
- method
- static method

Lock interface

synchronized

```java
class ClassName {
	public void method() {
		synchronized(ClassName.class) {
			// TODO
		}
	}
}

class ClassName {
	public void method() {
		synchronized(this) {
			// TODO
		}
	}
}

public class Apple{
	public synchronized void method() {
		// TODO
	}

	public synchronized static void method2() {

	}
}
```

```
public class Main {
	public static void main(String[] args) {
		Apple app = new Apple();
		app.method();

		Apple app1 = new Apple();
		app1.method();

		Apple.method2();
	}
}
```

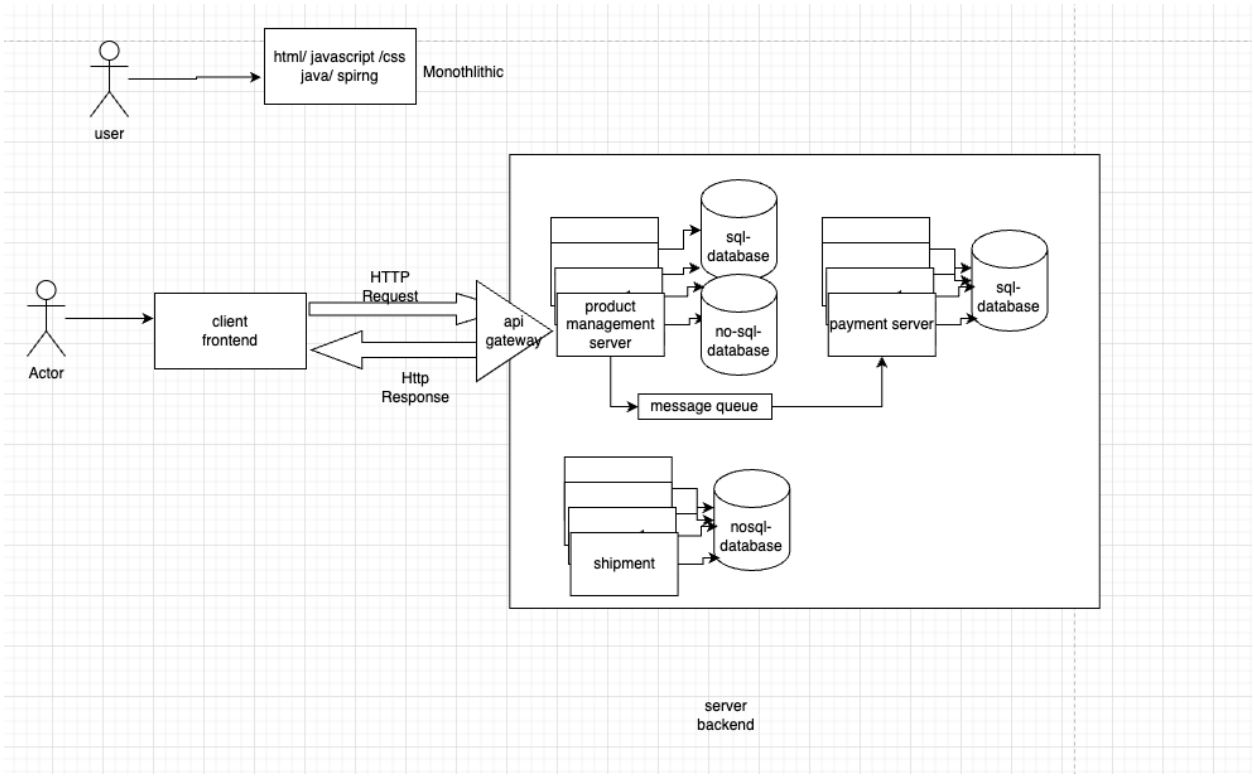
Lock interface
- lock(), unlock(), tryLock(), newCondition(), lockInterruptibly();

ReentrantLock class: lock on a resources more than once
ReadWriteLock Interface
- Lock readLock();
- Lock writeLock();

DeadLock: two or more threads are blocked forever
How to avoid the deadlock,
1. remove unnecessary lock
2. don't use nested lock
3. join()

Web Basics

1. Web Architecture



client - server model
application service
http request/response
horizontal scaling vs vertical scaling
load balancer
microservice, microfronted
database: relational database (sql database), nonrelational database (no-sql database),
api gateway
message queue
log, monitor
deployment with AWS/Azure/GCP
security (authentication and authorization)
why testing

2. Design Pattern

23 types
Creational Pattern

- singleton pattern
- factory pattern
- builder pattern
- …

Structural Pattern

- adaptor pattern
- decorator pattern
- proxy pattern (static and dynamic pattern)
- compositor pattern
- …

Behavioral Pattern

- Observer pattern
- iterator Pattern
- Visitor pattern
- Mediator pattern
- ….

singleton pattern (know how to code the double check locking version)
factory pattern
builder pattern
proxy pattern (static and dynamic pattern)

Singleton Pattern

only one object

Use case

- Hardware interface access
- Logger
- Configuration File
- thread pool
- caching
- …

```java
// 1. eager loading
public class SingletonEx {
    private static final SingletonEx instance = new SingletonEx();

    private SingletonEx() {
        if (instance != null) {
            throw new RuntimeException("can not create");
        }
    }

    public static SingletonEx getInstance() {
        return instance;
```

```
    }

    // TODO


}


// 2. lazy loading, data race problem, not thread safe
public class SingletonEx {
    private static SingletonEx instance;

    private SingletonEx() {
        if (instance != null) {
            throw new RuntimeException("can not create");
        }
    }

    public static SingletonEx getInstance() {
        // thread1, thread2
        if (instance == null) {
            instance = new SingletonEx();
        }
        return instance;
    }

    // TODO


}


// 3. synchronized, only one thread can access, not efficient
public class SingletonEx {
    private static SingletonEx instance;

    private SingletonEx() {
        if (instance != null) {
            throw new RuntimeException("can not create");
        }
    }

    // thread1, thread2
    public static synchronized SingletonEx getInstance() {
        if (instance == null) {
```

```
            instance = new SingletonEx();
        }
        return instance;
    }

    // TODO


}

-------------  pick one version below to understand and memorize --------------

// 4. synchronized, double check locking mechanism
public class SingletonEx {
    private volatile static SingletonEx instance;

    private SingletonEx() {
        if (instance != null) {
            throw new RuntimeException("can not create");
        }
    }



    public static SingletonEx getInstance() {

        if (instance == null) {
            // thread1, thread2
            synchronized (SingletonEx.class) {
                if (instance == null) {
                    instance = new SingletonEx();
                    // 1. allocate memory to instance
                    // 2 new singleton()
                    // 3. assign singleton object to instance
                }
            }
        }

        return instance;
    }

    // TODO
}

 class LazySingleton {
```

```
    // The instance is not created at class loading time
    private static class SingletonHelper {
        private static final LazySingleton INSTANCE = new LazySingleton();
    }

    // Private constructor to prevent external instantiation
    private LazySingleton() {

    }

    // Public method to provide a global access point, which only loads the
    // SingletonHelper class (and thus creates the instance) when called
    public static LazySingleton getInstance() {
        return SingletonHelper.INSTANCE;
    }
}

// Enum to create singleton
public enum SingletonEx{
    INSTANCE;

    // more logic
}

public static void main(String[] args) {
    SingletonEx singleton = SingletonEx.INSTANCE;
}
```

Builder Patter

Car myCar = Car.builder().frame(“iron”),make(“Tesla”).model(“Y”).modleYear(2025).build();

Department dp = Department.builder().setName(“name”).setId(123).build();
Department dp = Department.builder().setName(“name”).build();

```
Java
package org.example.builder_pattern;
//
//public class Department {
```

```
//    private String name;
//    private String location;
//    private String managerName;
//    private Integer Id;
//
//    public Department() {
//    }
//
//    public Department(String name) {
//        this.name = name;
//    }
//
//    public Department(String name, String location) {
//        this.name = name;
//        this.location = location;
//    }
//
//    public Department(String name, String location, String managerName) {
//        this.name = name;
//        this.location = location;
//        this.managerName = managerName;
//    }
//
//    public Department(String name, String location, String managerName, Integer
id) {
//        this.name = name;
//        this.location = location;
//        this.managerName = managerName;
//        Id = id;
//    }
//}

public class Department {
    private String name;
    private String location;
    private String managerName;
    private Integer id;

    public Department(String name, String location, String managerName, Integer
id) {
        this.name = name;
        this.location = location;
        this.managerName = managerName;
        this.id = id;
    }
```

```
    public static DepartmentBuilder builder(){
        return new DepartmentBuilder();
    }

    public static class DepartmentBuilder{
        private String name;
        private String location;
        private String managerName;
        private Integer id;

        public DepartmentBuilder setName(String name) {
            this.name = name;
            return this;
        }
        public DepartmentBuilder setLocation(String location) {
            this.location = location;
            return this;
        }
        public DepartmentBuilder setManagerName(String managerName) {
            this.managerName = managerName;
            return this;
        }
        public DepartmentBuilder setId(Integer id) {
            this.id = id;
            return this;
        }
        public Department build(){
            Department d = new Department(name, location, managerName, id);
            return d;
        }
    }
}
```

Factory
create object without expose the internal logic

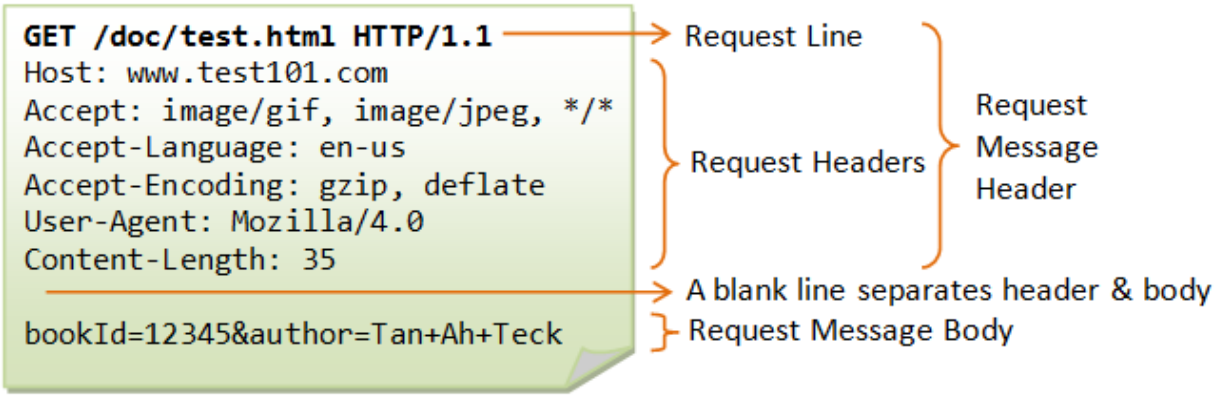
Java
```
public class Apple {
```

```
}


public class Orange {

}


public class FruitFactory {

  Map<String,Object> fruitMap = new HashMap<>();

  public FruitFactory(){
     fruitMap.put("apple", new Apple());
     fruitMap.put("orange", new Orange());
  }

  public Object createFruit(String name){
     return fruitMap.get(name);
  }

  public static void main(String[] args) {
     FruitFactory f = new FruitFactory();
     f.createFruit("apple");
     f.createFruit("orange");
  }
}
```

Proxy
static proxy
dynamic proxy

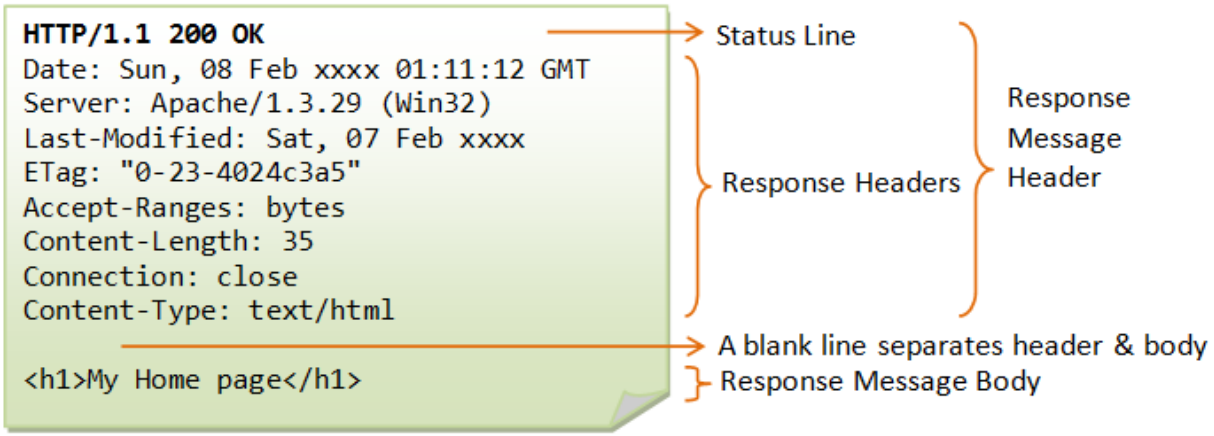
3. HTTP

http request



http method: get/put/post/delete/….

http response



Http status code

- 1xx Informational
- 2xx Success
- 3xx Redirection
- 4xx Client Error
- 5xx Server Error

200 OK
201 Created
202 Accepted
204 No Content

307 Temporary Redirect
308 Permanent Redirect

400 Bad Request

401 Unauthorized
403 Forbidden
404 Not Found

500 Internal Server Error

http methods

Method Name	Desc	Safe	Idempotent	Cacheable
get	read/ fetch	Yes	Yes	Yes
post	create	No	No	No
put	update	No	Yes	No
delete	remove	No	Yes	No
patch	partial update	No	No(Can Be)	No
….				

other methods: head, options, trace, connect

get www.mycompany.com/api/student/23
delete www.mycompany.com/api/student/23

don't put verb into url, like: www.mycompany.com/api/getstudents

safe: whether the method will alter the state of the server

idempotent: multiple identical requests → same effect (!= no effect)

post www.mycompany.com/api/student

```
{
        name: "Alice",
        Age: 20
}
```

put www.mycompany.com/api/student/1

```
{
        id: 1
        name: "Bob",
        Age: 30
```

}

delete www.mycompany.com/api/student/3

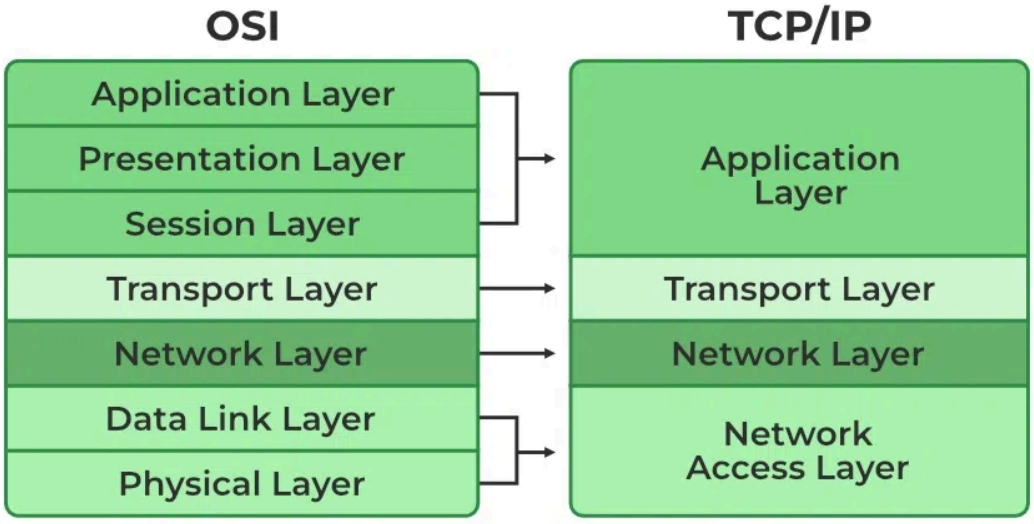
—---------- database —---------------

id	name	age
1	Bob	30
2	Alice	20

4. OSI model, TCP/IP model

OSI (Open Source Interconnection) 7 Layer Model

Layer	Application/Example		Central Device/ Protocols			DOD4 Model
Application (7) Serves as the window for users and application processes to access the network services.	**End User layer** Program that opens what was sent or creates what is to be sent Resource sharing • Remote file access • Remote printer access • Directory services • Network management		**User Applications** SMTP		GATEWAY	Process
Presentation (6) Formats the data to be presented to the Application layer. It can be viewed as the "Translator" for the network.	**Syntax layer** encrypt & decrypt (if needed) Character code translation • Data conversion • Data compression • Data encryption • **Character Set Translation**		JPEG/ASCII EBDIC/TIFF/GIF PICT			
Session (5) Allows session establishment between processes running on different stations.	**Synch & send to ports** (logical ports) Session establishment, maintenance and termination • Session support - perform security, name recognition, logging, etc.		**Logical Ports** RPC/SQL/NFS NetBIOS names			
Transport (4) Ensures that messages are delivered error-free, in sequence, and with no losses or duplications.	**TCP** Host to Host, Flow Control Message segmentation • Message acknowledgement • Message traffic control • Session multiplexing	PACKET FILTERING	TCP/SPX/UDP			Host to Host
Network (3) Controls the operations of the subnet, deciding which physical path the data takes.	**Packets** ("letter", contains IP address) Routing • Subnet traffic control • Frame fragmentation • Logical-physical address mapping • Subnet usage accounting		**Routers** IP/IPX/ICMP		Can be used on all layers	Internet
Data Link (2) Provides error-free transfer of data frames from one node to another over the Physical layer.	**Frames** ("envelopes", contains MAC address) [NIC card — Switch — NIC card] (end to end) Establishes & terminates the logical link between nodes • Frame traffic control • Frame sequencing • Frame acknowledgment • Frame delimiting • Frame error checking • Media access control		**Switch Bridge WAP** PPP/SLIP	Land Based Layers		Network
Physical (1) Concerned with the transmission and reception of the unstructured raw bit stream over the physical medium.	**Physical structure** Cables, hubs, etc. Data Encoding • Physical medium attachment • Transmission technique - Baseband or Broadband • Physical medium transmission Bits & Volts		**Hub**			

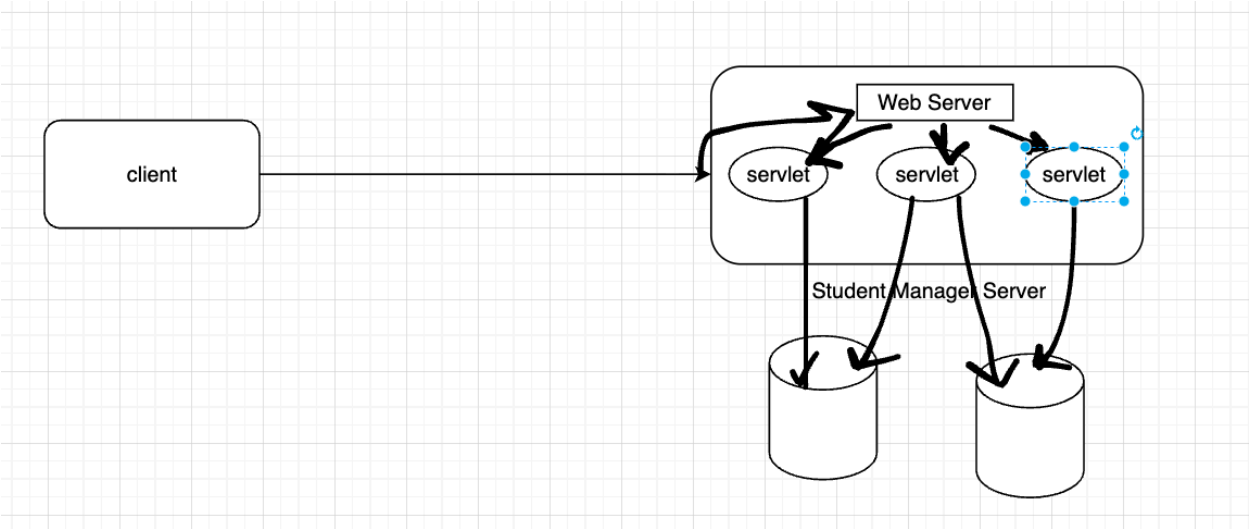


TCP vs UDP

TCP 3 way handshaking

TCP 4 way handshaking
Application layer: Http, FTP, SMTP

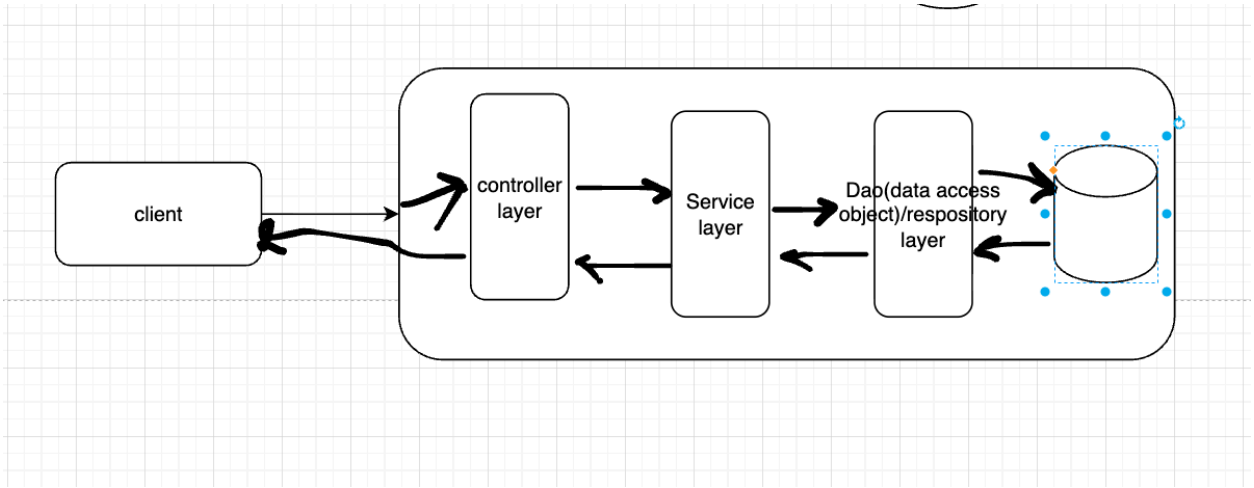
5. Java Web Server



web server

- Tomcat
- Jboss
- Jetty
- Apache TomEE
- Weblogic
- Webspere
- ,,,

6. Three-Tier architecture



controller: handle all the http requests
service: process business logic
DAO: connect with database

product management server:
- ProductController:
  - get api/product
  - post api/product
  - …
- ProductService:
  - call the repository layer
  - add some description into the product
  - calculate the total price
- ProductRepository:
  - connect with database
  - write some query

Spring

java -> Spring
python -> Django
javascript/typescript -> React/ Angular
C# -> .NET Core/ ASP .NET

1. IOC

Inversion of Control

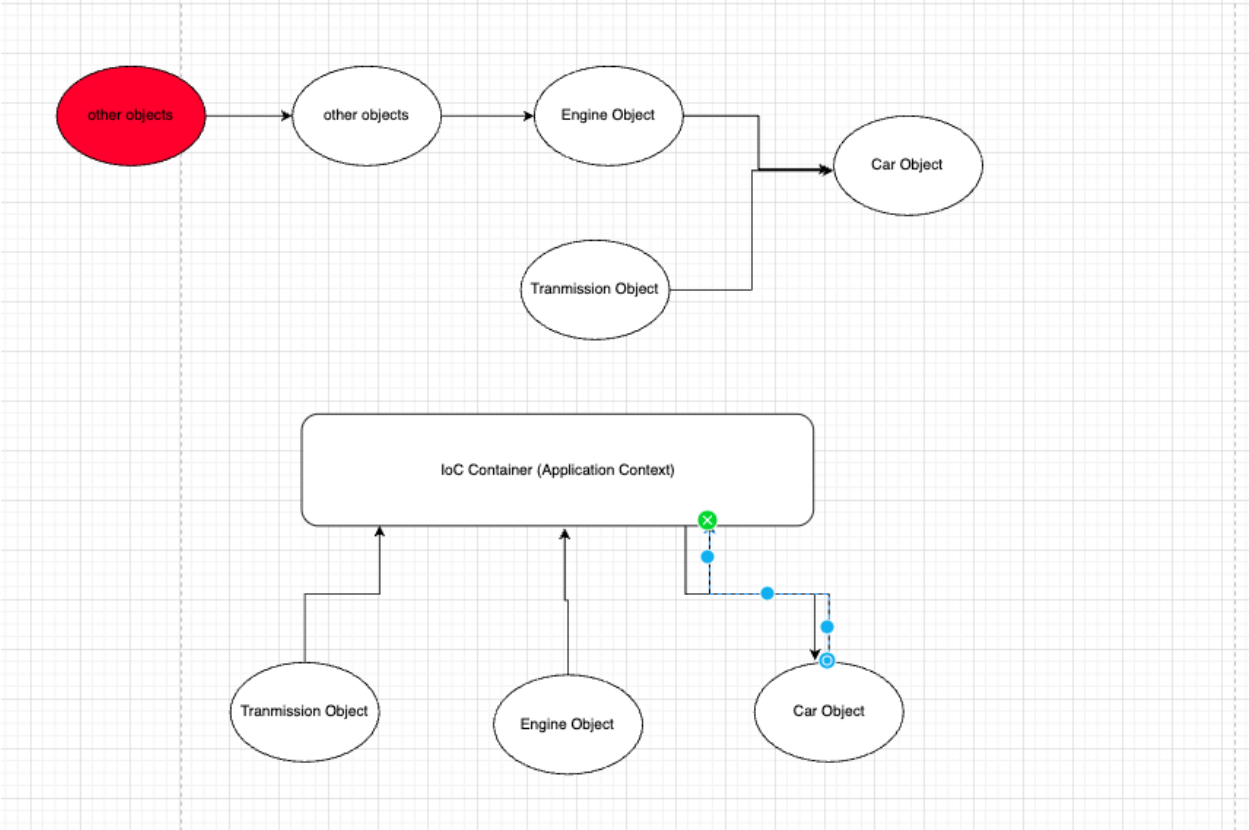
```
class Car {
	Engine engine;
	Transmission transmission;

	Car ()  {
		this.engine = new Engine(....);
		this.transmission = new Transmission(...);
	}


}

class Engine {

}

class Transmission {

}

Car car = new Car();
```

.jar/ .war

company.com

com.company

@Bean vs @Component
@Autowired
@Configuration
@ComponentScan



2. DI

3 types of DI

- Constructor Based DI
- Setter Based DI
- Field Based DI

pros and cons for 3 types of DI

3. Bean Scope

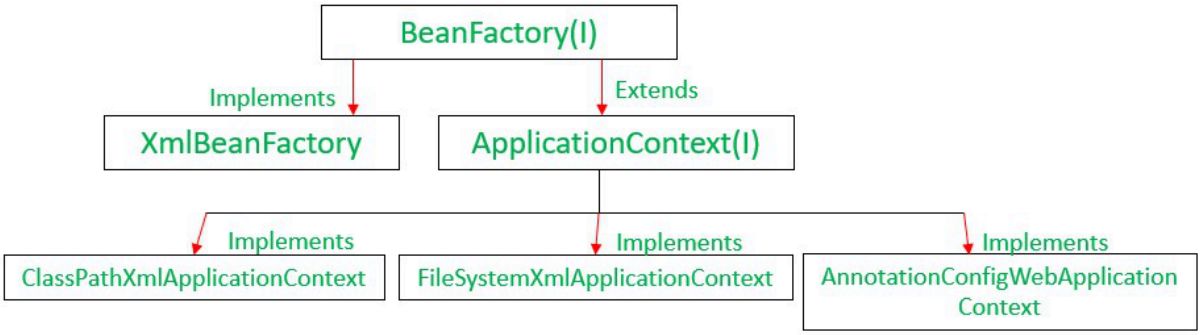
- Singleton (default): single bean object instance per ioc container
- prototype: create new object for each time of dependency injection
- request: per request
- session: per session
- application: per ServerletContext

- websocket: Websocket -> cusomize

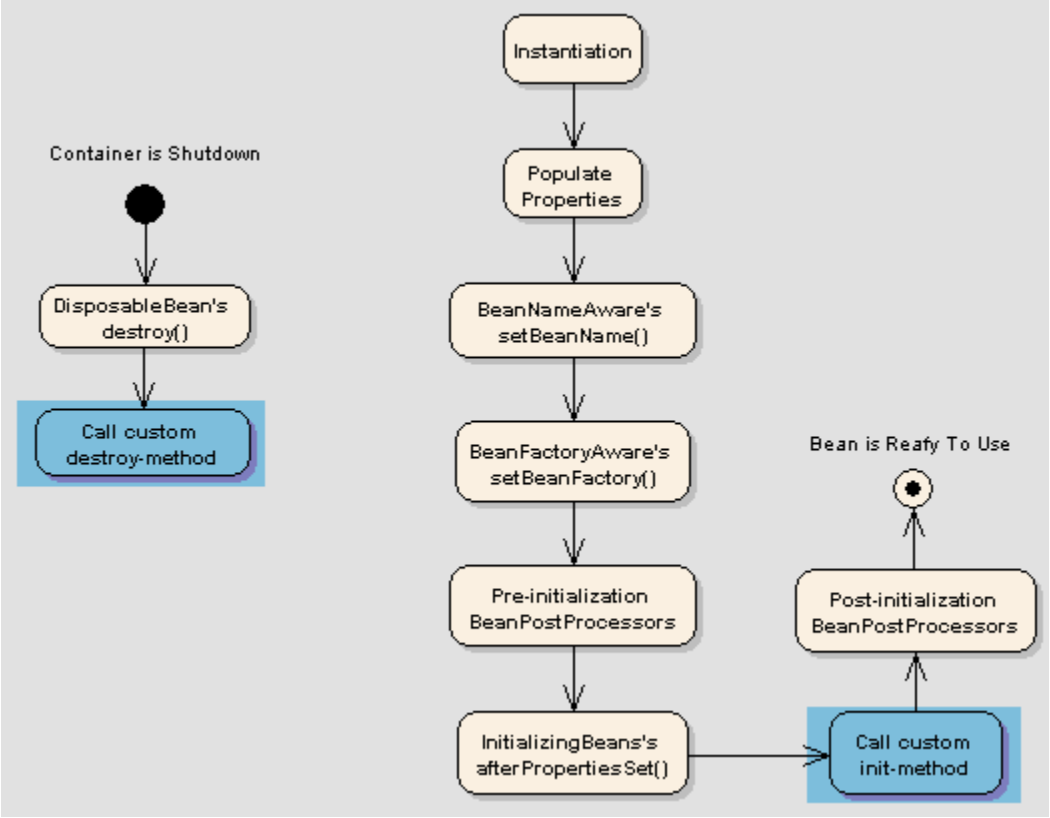
@Scope("prototype")
@Bean

singleton: 1 bean per IOC Container/ Application Context
application: 1 bean per ServletContainer

4. IOC Container/ ApplicationContext/ BeanFactory



5. BeanLifeCycle




```java
@Service
public class CustomerService {
	// Business Logic about Customer
}
```

Real World Example

```java
@Servie
public class S3JobSchedulerService {
	private ExecutorService executor;

	@PostConstruct
	public void startExecutor() {
		exector = Exectuors.newFixedThreadPool(6);
		System.out.println("Thread Pool Started")
	}
```

```
        @PreDestroy
        public void stopExecutor() {
               executor.shutdown();
               System.out.println("Thread Pool shut down");

        }

}
```

6. AOP

Logger log = new …

public class CustomerService {
	public void method1() {
		// logic

	}

	public void method2() {
		// logic

	}
}

public class CustomerService2 {

	public void method1() {
		// logic

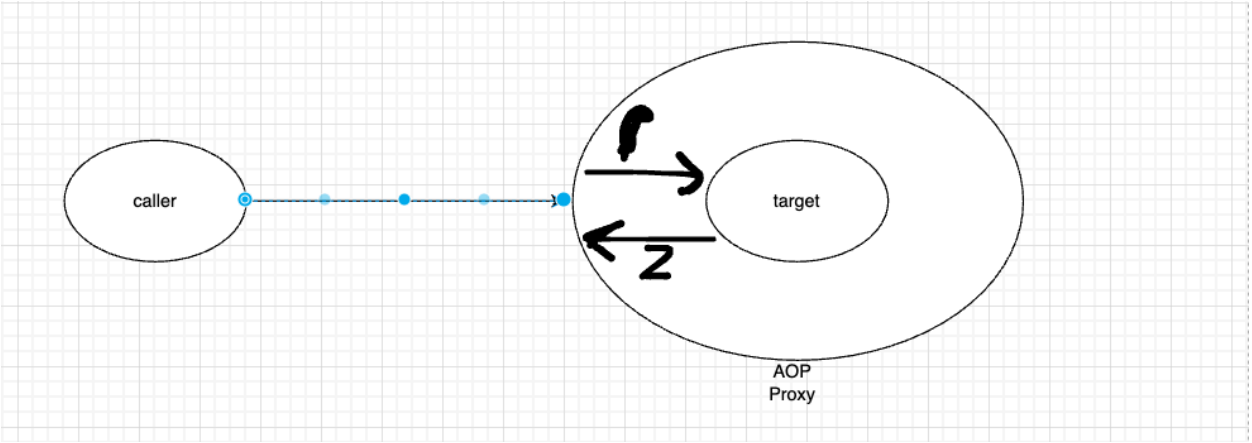
	}

	public void method2() {
		// logic
	}
	}

AOP

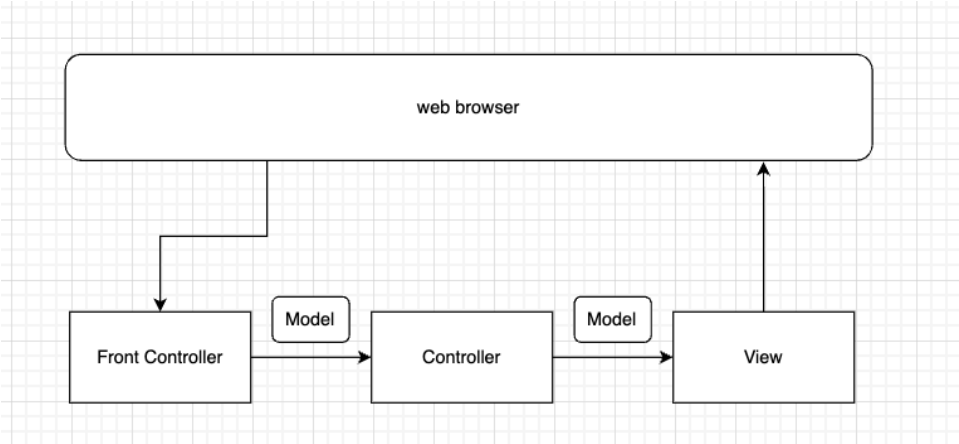
- aspect
- advice: before, after return, after throw, after, around
- joinpoint
- pointcut
- target

internal logic for AOP

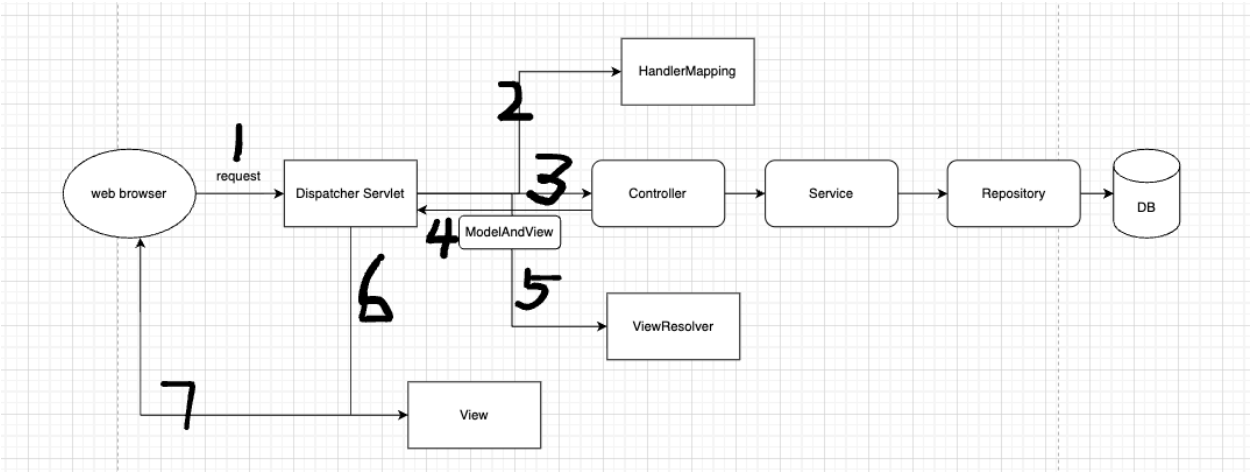


7. Spring MVC

spring core (xml, severlet) -> spring mvc (DispatcherSeverlet) -> Spring boot



Spring MVC working flow



8. Spring boot

How to create a spring boot project?

https://start.spring.io/

Why Spring Boot?

- **spring boot starter**
- **auto-configuration**
- **embedded Server -> Tomcat**
- **spring boot actuator**
- **annotation based,**
- **….**

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-validation</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

```
@Target({ElementType.TYPE})
```

```
@Retention(RetentionPolicy.RUNTIME)
@Documented
@Inherited
@SpringBootConfiguration
@EnableAutoConfiguration
@ComponentScan(
   excludeFilters = {@Filter(
   type = FilterType.CUSTOM,
   classes = {TypeExcludeFilter.class}
), @Filter(
   type = FilterType.CUSTOM,
   classes = {AutoConfigurationExcludeFilter.class}
)}
)
public @interface SpringBootApplication {}
```

http url design

www.rothurtech.com/…./api /users/id

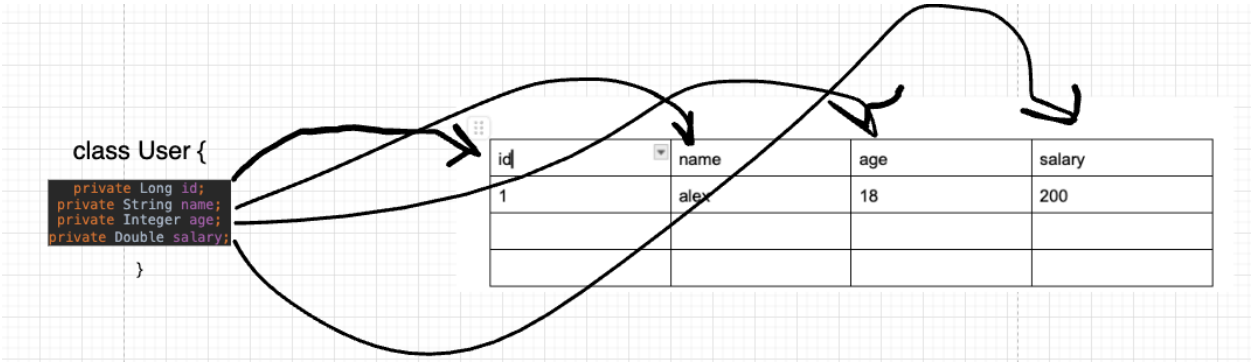
GET api/users
GET api /users/id
PUT api/users/id
POST api/users
DELETE api/users/id

user has multiple email accounts:
GET api /users/2/emailaccounts
GET api /users/2/emailaccounts/3

sort the request
GET api/users?sort=priority,-create_at
request specific fields
GET api/users?fields=id,name

GET api/users?sort=priority,-create_at&fields=id,name

Spring Data JPA: ORM (object relational mapping) -> Hinerbate -> JDBC



javaEE, 13 specifications, JAVA JPA, EntityManager
Hibernate/ Mybatis…

Session/ SessionFactory -> Entity

Spring Data JPA ->

@RestController = @Controller + **@ResponseBody**

@RequestBody: json -> java object
@ResponseBody:java object -> json

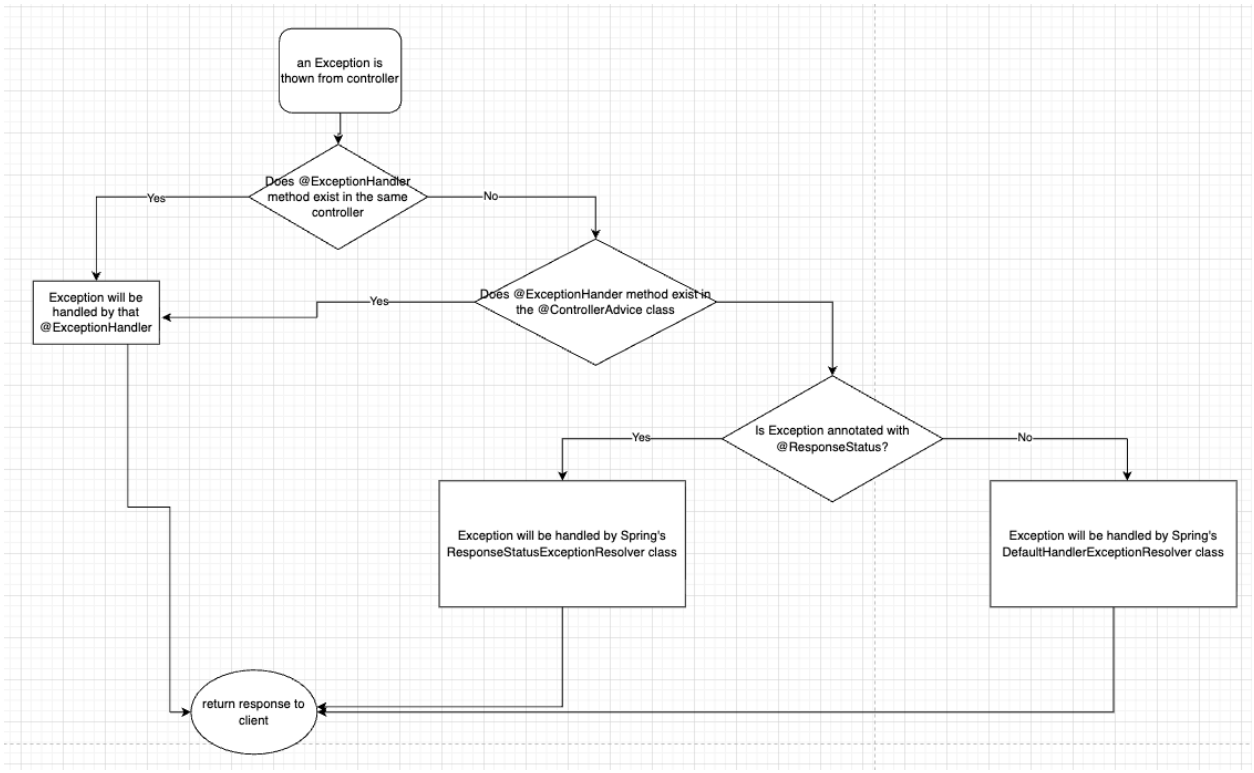
GET api/users?sort=salary

@RequestParam:

GET api/users/{id}
@PathVariable:

GET api /users/{id}/emailaccounts? fields=address

Exception Handling
java: try catch, throws



`ResponseStatus`
`ExceptionHandler`
`ControllerAdvice`

`Validation`
`min/max/notblank, pattern, email`….

Database

1. Relational Database (sql database)

database:
DBMS (database management system): MySQL, PostgreSQL, SQL Server, Oracle etc

SQL (Structured Query Language): query language, minor syntax changes amount different databases

- insert data into a table
- select data from table

for example

- MySql
    - SELECT CHARACTER_LENGTH(data) FROM TableName
- MS SQL Server
    - SELECT LEN(data) FROM TableName

Mysql
get row 21 - 30, order by name

```SQL
select *
from tablename
order by name
LIMIT 20, 10
```

Oracle

```SQL
select *
from tablename
order by name
OFFSET 20 ROWS FETCH NEXT 20 ROWS ONLY;
```

id	name	age	email	address
1	Bob	20	bob@gmail.com	xxxx CA 12012

2. Database Normalization

total 6, achieve best practice at 3rd

1NF (first normal form):

id	name	age	email	address
1	Bob, Jerry	20, 30	bob@gmail.com	xxxx CA 12012

2NF

- be in 1NF
- single column primary key

id	firstname	secondname	phone	department
1	A	B	123456	java
2	A	C	123456	Java
3	D	C	123456	Java

3NF

- be in 2NF
- has no transitive functional dependencies

table1

id	birth year	
1	2001	
2	1995	

table2

id	age
1	25
2	31

select *....
join table1 and table2

de-normalization

3. Non-relational Database (no-sql database)

- document based data store

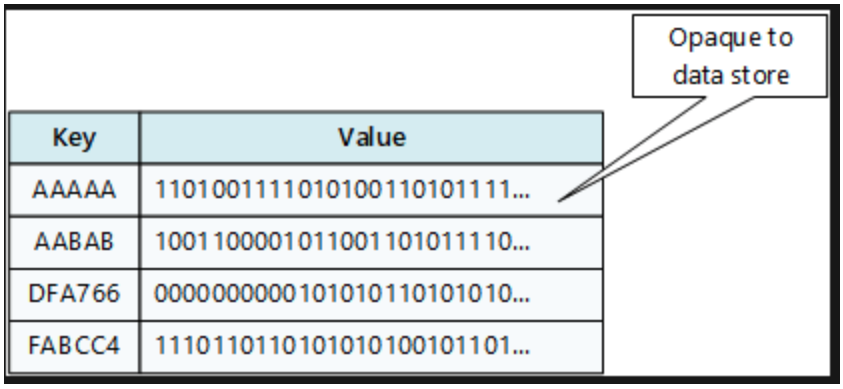
Key	Document
1001	{ "CustomerID": 99, "OrderItems": [{ "ProductID": 2010, "Quantity": 2, "Cost": 520 }, { "ProductID": 4365, "Quantity": 1, "Cost": 18 }], "OrderDate": "04/01/2017" }
1002	{ "CustomerID": 220, "OrderItems": [{ "ProductID": 1285, "Quantity": 1, "Cost": 120 }], "OrderDate": "05/08/2017" }

- Columnar based Data Stores (column family)

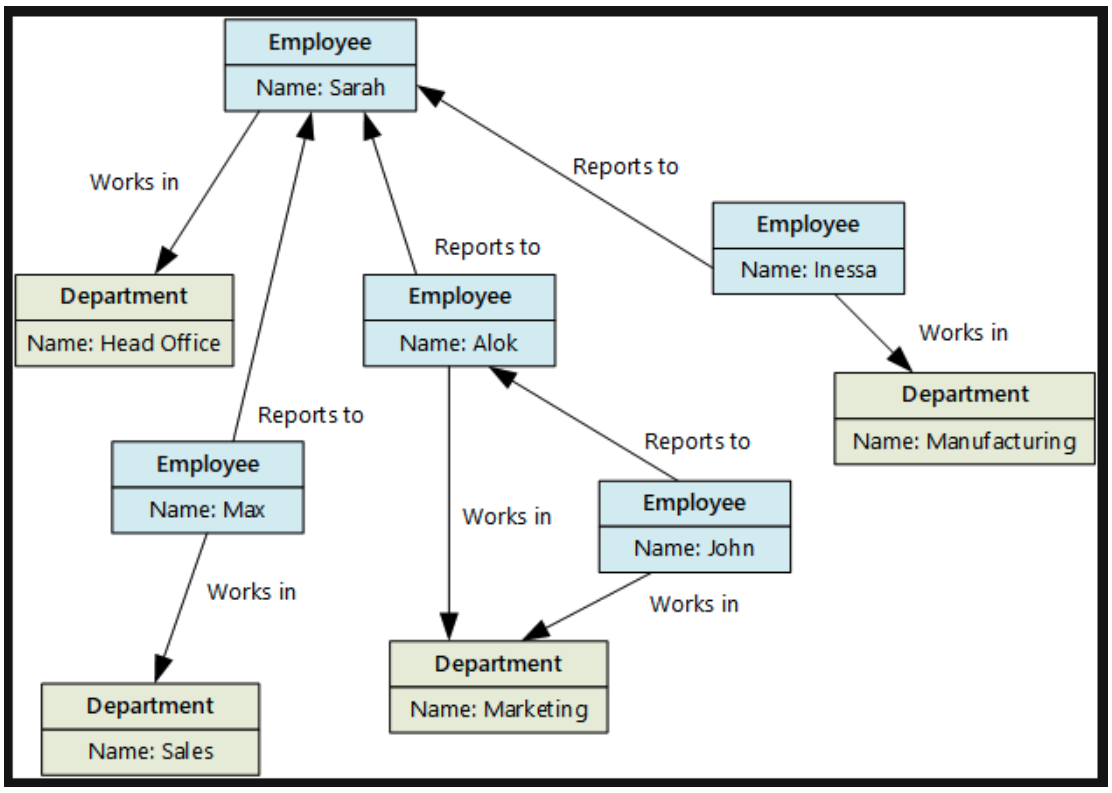
CustomerID	Column Family: Identity
001	**First name:** Mu Bae **Last name:** Min
002	**First name:** Francisco **Last name:** Vila Nova **Suffix:** Jr.
003	**First name:** Lena **Last name:** Adamcyz **Title:** Dr.

CustomerID	Column Family: Contact Info
001	**Phone number:** 555-0100 **Email:** someone@example.com
002	**Email:** vilanova@contoso.com
003	**Phone number:** 555-0120

- Key/Value based data store



Graph Data store

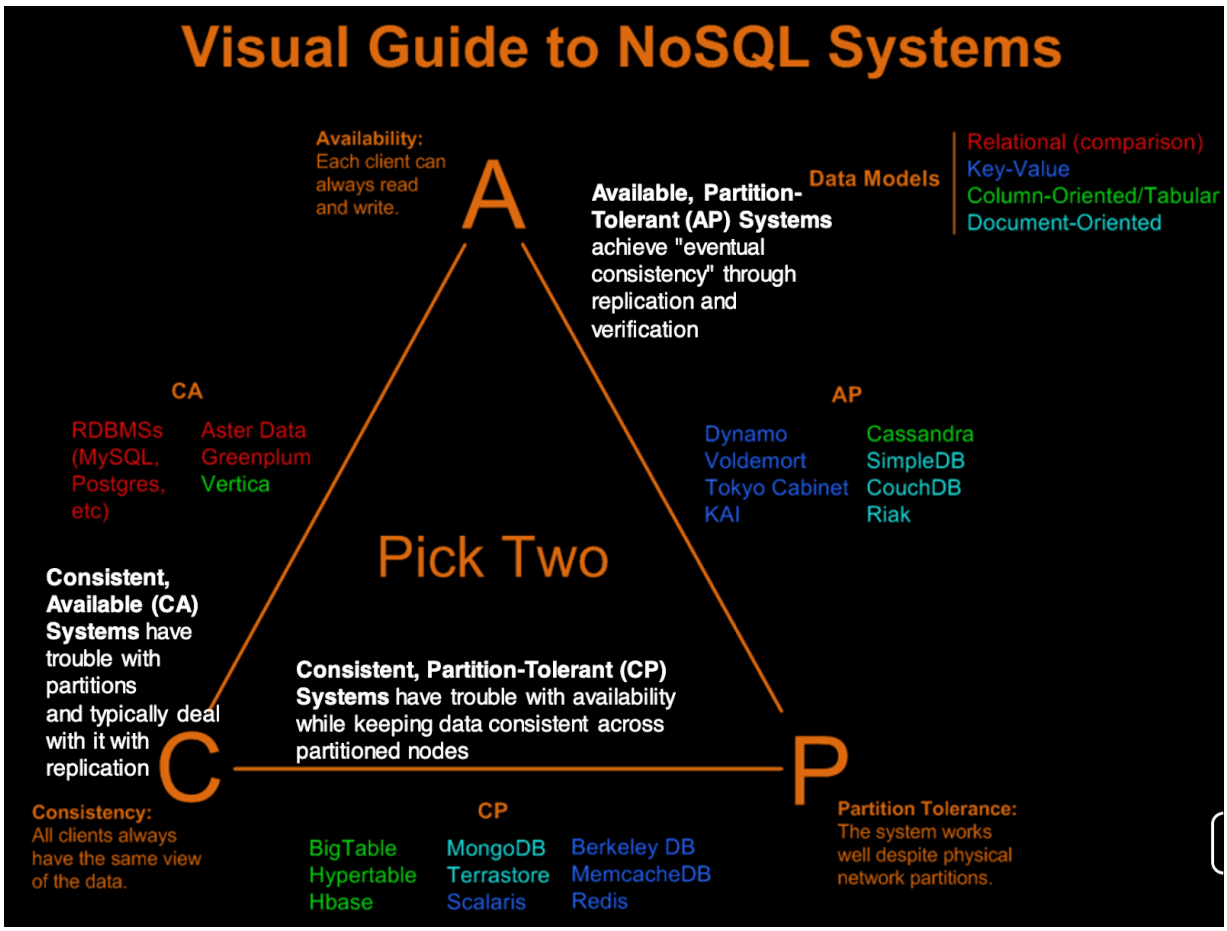


examples:

Document: mongoDB, CouchDB
Column: Cassandra, HBase
Key-value: redis, Riak
Graph database: Neo4j, GraphDB

4. Theory of Nosql: CAP

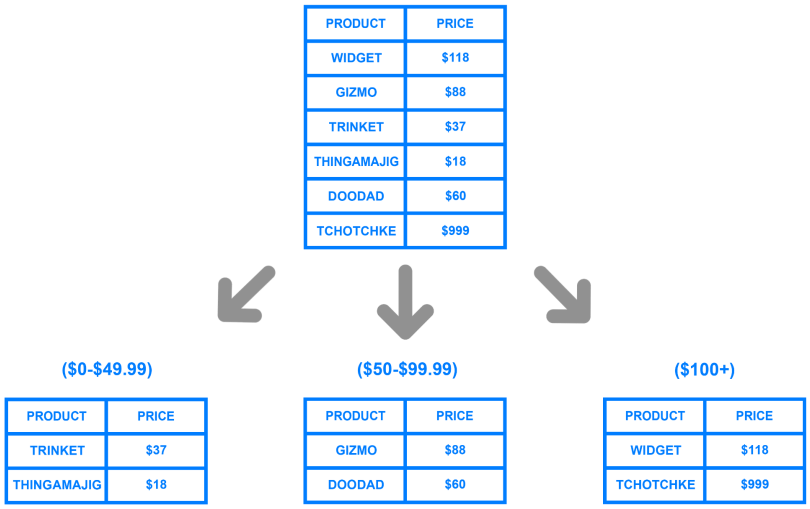
CAP:
Consistency
Availability:
Partition Tolerance:



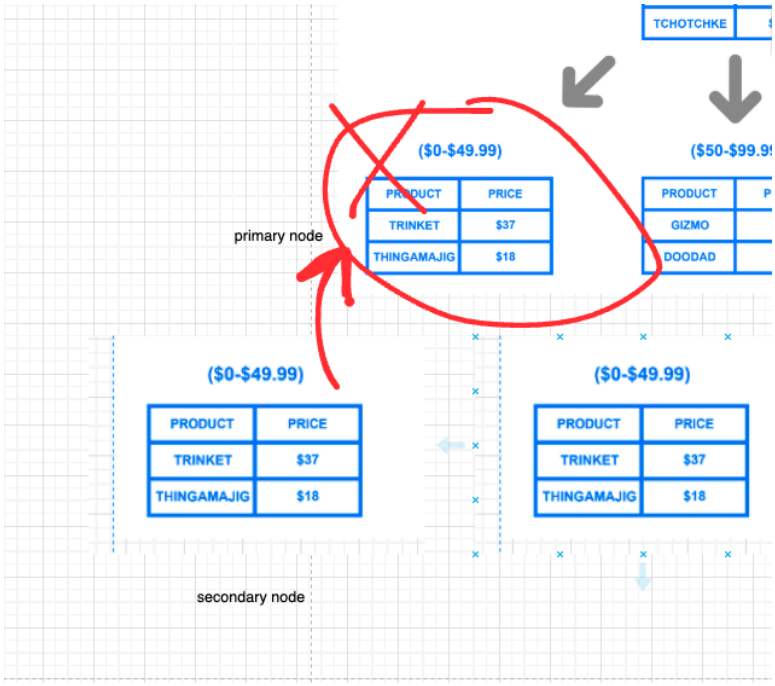
AP + eventually consistence

5. Sharding vs Replica

sharding:



replica



6. sql vs no-sql database

sql	nosql

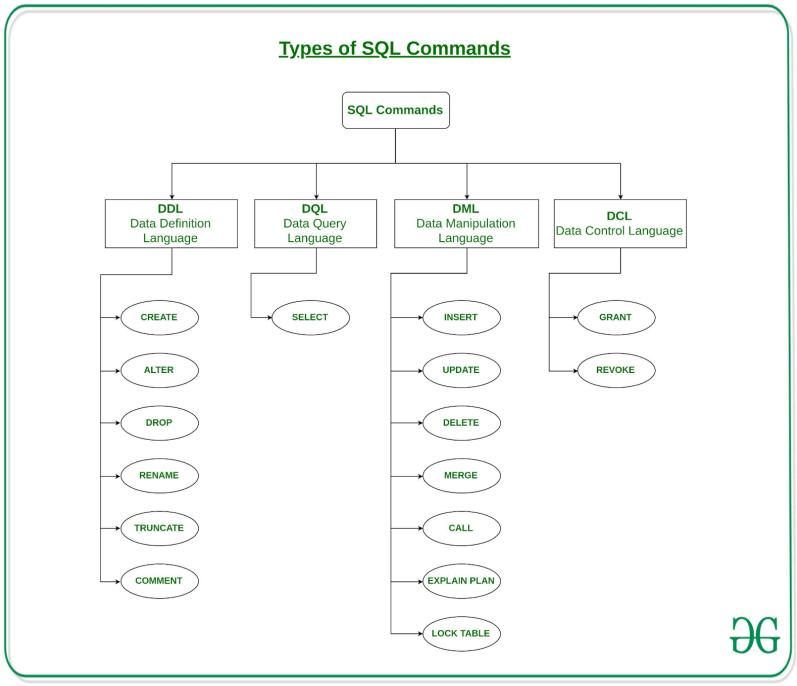
relational database	non-relational database
predefined schema	dynamic schema
vertical scaling	horizontal scaling
ACID	CAP
slower (index)	faster (10x)

ACID

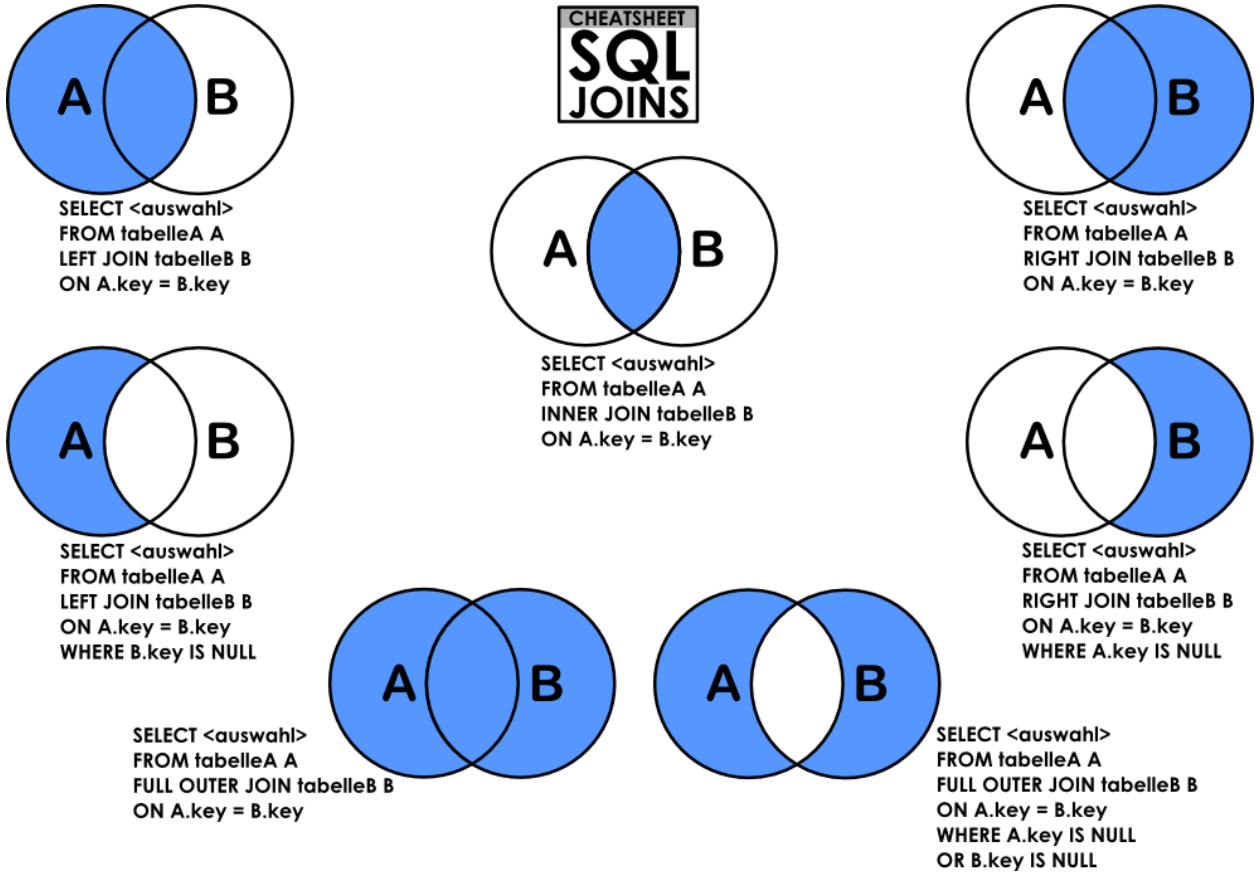
Index: clustered index/ non-clustered index

7. Basic SQL language

DDL: data definition language: CREATE, DROP, TRUNCATE …
DML: data manipulation language: INSERT, UPDATE, DELETE…
DQL: data query language, SELECT FROM, GROUP BY….
DCL: data control language: GRANT, REVOKE
DTL: data transaction language, COMMIT, ROLLBACK, SAVEPOINT



inner join, left join, right join, outer join, cross join



cross join

table1
A
B
C

table2
C
D

AC
AD

BC
BD
CC
CD

dense_rank() vs rank()

id	name	age	dense_rank()	rank()
1	A	10	1	1
2	B	20	2	2
3	C	20	2	3

```sql
-- create

CREATE TABLE department (
    dept_id INT PRIMARY KEY AUTO_INCREMENT,
    dept_name VARCHAR(50) NOT NULL
);

CREATE TABLE employee (
    emp_id INT PRIMARY KEY AUTO_INCREMENT,
    emp_name VARCHAR(50) NOT NULL,
    age INT NOT NULL,
    salary DECIMAL(10,2),
    dept_id INT,
    FOREIGN KEY (dept_id) REFERENCES department(dept_id)
);



-- insert
INSERT INTO department (dept_name) VALUES
('Engineering'),
('HR'),
('Finance'),
('Marketing'),
('Compliance');
```

```sql
INSERT INTO employee (emp_name, age, salary, dept_id) VALUES
('Alice', 25, 85000.00, 1),
('Bob', 35, 95000.00, 1),
('Charlie', 30, 60000.00, 2),
('Diana', 42, 110000.00, 3),
('Ethan', 25, 110000.00, 4),
('Fiona', 32, 72000.00, 2),
('George', 45, 120000.00, 3);

-- fetch
SELECT * FROM employee;
SELECT * FROM department;

-- Find employees whose salary is greater than 80,000.
select *
from employee
where salary > 80000;

-- Get the youngest employee.
select *
from employee
order by age ASC
limit 1;

-- what if multiple answers
select *
from employee
where age = (
  select min(age)
  from employee
);

-- Count employees in each department.
select
  d.dept_name,
  count(e.emp_id) AS employee_count
from department d
left join employee e
  on d.dept_id = e.dept_id
group by d.dept_id;

-- List employees who do not belong to any department
select e.*
from employee e
left join department d
```

```sql
  on e.dept_id = d.dept_id
where d.dept_id IS NULL;

-- Find employees earning the second highest salary.
select *
from employee
order by salary desc
limit 1 offset 1;

select *
from employee
where salary = (
  select max(salary)
  from employee
  where salary < (
    select max(salary)
    from employee
  )
);

select *
from (
  select *,
      DENSE_RANK() over (order by salary desc) as rnk
      from employee
) t
where rnk = 2;
```

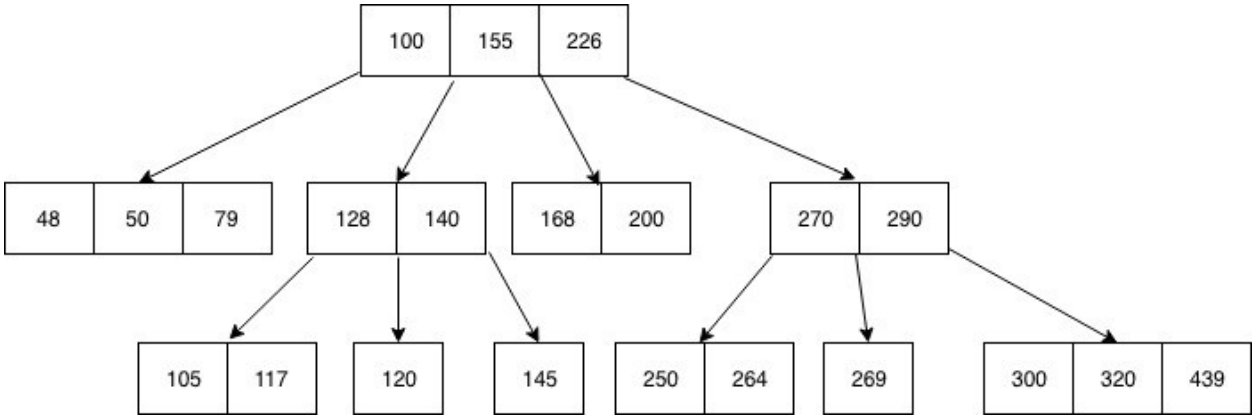
8. Index

Clustered Index
Non-clustered Index

id	name	age	email
1			
2			

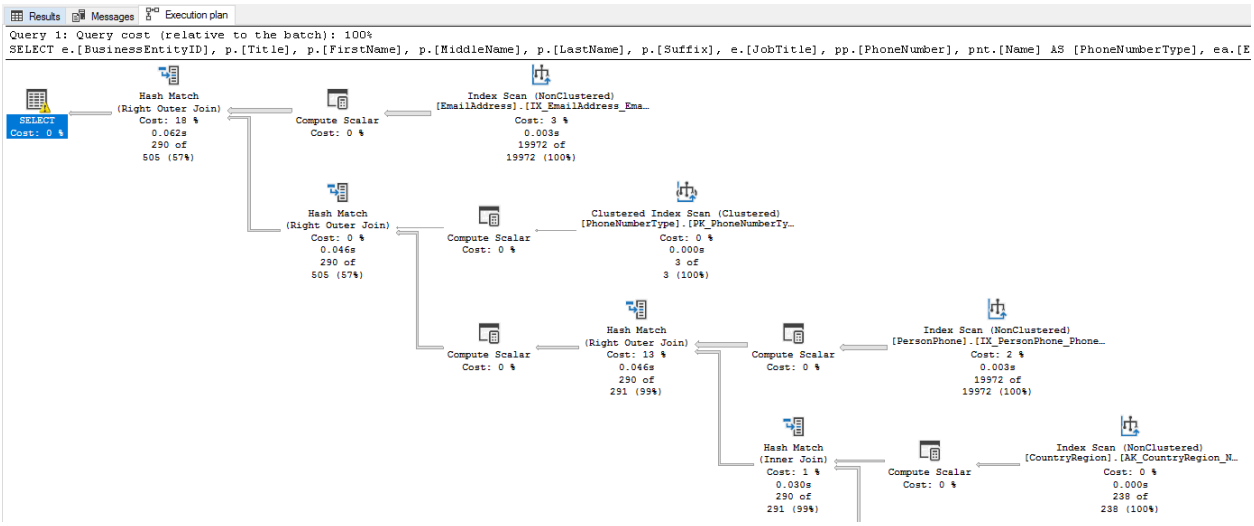
index data structure: (innoDB)
b tree

b+ tree
hash table
R tree
k-d tree
bitmap



9. SQL Tuning(sql slow)

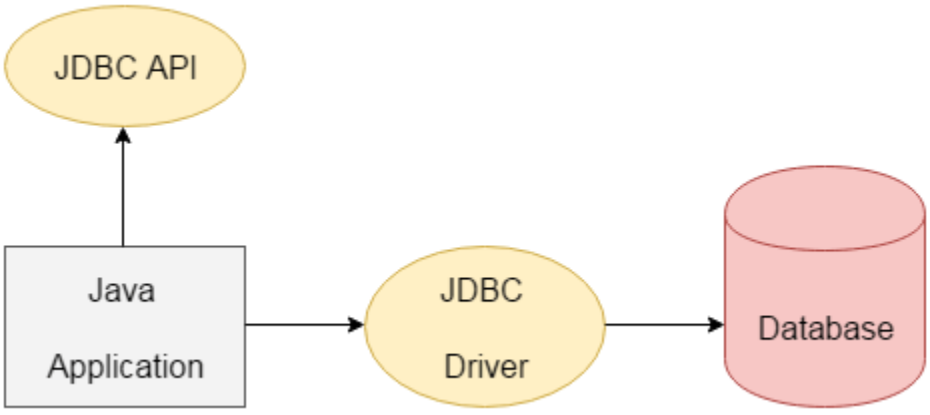
1. Using Execution Plan to identify the cause of slowness.
2. try to reduce joins. Remove unused join and join conditions.
3. Using index to improve join. Make sure indexes are used in the join query.
4. Use UNION ALL instead of UNION. Because Union remove duplicates.
5. Select exact fields instead of *
6. Use "Join On" instead of "where" to join tables.
7. Reduce the usage of LIKE "%ABC%", especially the "%" in the front
8. Use LIMIT to do pagination if the data is huge.
9. Create View or stored procedure to improve performance.
10. Avoid using IN. (full table scan)
11. Change Sub Query to join. Or Join to Sub Query. Sometimes one is faster than the other.



JDBC & Hibernate

1. JDBC

java database connectivity



Statement vs prepared statement vs callable statement

SQL injection

Java

Selection

```java
package jdbcDemo;

import jdbcDemo.utils.JdbcConfig;

import java.sql.*;

public class JdbcSelectTest2 {

    public static void main(String[] args) {
       Connection conn = null;
       Statement stmt = null;

        try {
            conn = DriverManager.getConnection(
                    JdbcConfig.getUrl(),
                    JdbcConfig.getUser(),
                    JdbcConfig.getPassword()
            );
            stmt = conn.createStatement();

            String strSelect = "select title, price, qty from books";

            System.out.println("The sql query is " + strSelect);
            System.out.println();

            ResultSet resultSet = stmt.executeQuery(strSelect);
            System.out.println("The records selected are:");
            int rowCount = 0;


            while (resultSet.next()) {
                String title = resultSet.getString("title");
                String price = resultSet.getString("price");
                int qty = resultSet.getInt("qty");
                System.out.println(title + ", " + price + ", " + qty);
                rowCount++;
            }
            System.out.println("Totel number of records = " + rowCount);

        } catch (SQLException throwables) {
            throwables.printStackTrace();
```

```
        } finally {
            try {
                if (stmt != null) stmt.close();
                if (conn != null) conn.close();

            } catch (SQLException throwables) {
                throwables.printStackTrace();
            }

        }
    }
}
```

Insert & Update

```
package jdbcDemo;

import jdbcDemo.utils.JdbcConfig;

import java.sql.*;

public class JdbcInsertAndDeleteTest {
    public static void main(String[] args) {
        try(
                Connection conn  = DriverManager.getConnection(
                        JdbcConfig.getUrl(),
                        JdbcConfig.getUser(),
                        JdbcConfig.getPassword()
                );
                Statement stmt = conn.createStatement();
        ){
            System.out.println("---------delete-----------");
            String sqlDelete = "delete from books where title = 'Java'";
            int countDelete = stmt.executeUpdate(sqlDelete);
            System.out.println(countDelete + " records are deleted");


            System.out.println("---------insert one record-----------");
            String sqlInsert = "insert into books" +
                    " values ('Math', 100, 4)";
            int countInsert = stmt.executeUpdate(sqlInsert);
            System.out.println(countInsert + " records are inserted");
```

```
            System.out.println("---------insert multiple records-----------");
            String sqlMultiInsert = "insert into books values " +
                    "('English', 123, 4), "
                    + "('Net', 321, 5)";
            int countMultiInsert = stmt.executeUpdate(sqlMultiInsert);
            System.out.println(countMultiInsert + " records are inserted");


            System.out.println("---------partial insert-----------");
            String sqlPartialInsert = "insert into books (title, price)" +
                    "values ('Data', 458)";
            int countPartialInsert = stmt.executeUpdate(sqlPartialInsert);
            System.out.println(countPartialInsert + " records are inserted");




            String sqlSelect = "select * from books";
            ResultSet resultSet = stmt.executeQuery(sqlSelect);

            while (resultSet.next()) {
                String title = resultSet.getString("title");
                String price = resultSet.getString("price");
                int qty = resultSet.getInt("qty");
                System.out.println(title + ", " + price + ", " + qty);

            }

        } catch (SQLException throwables) {
            throwables.printStackTrace();
        }
    }
}
```

PreparedStatement

```
package jdbcDemo;

import jdbcDemo.utils.JdbcConfig;

import java.sql.*;
```

```
public class JdbcPreparedStatementTest {
    public static void main(String[] args) throws SQLException {
        try (
                Connection conn = DriverManager.getConnection(
                        JdbcConfig.getUrl(),
                        JdbcConfig.getUser(),
                        JdbcConfig.getPassword()
                );
                PreparedStatement pstmt = conn.prepareStatement(
                        "insert into books values (?, ?, ?)"
                );
                PreparedStatement pstmtSelect = conn.prepareStatement("select *
from books");
        ) {
            // set all the parameters
           pstmt.setString(1, "Python");
           pstmt.setInt(2, 100);
           pstmt.setInt(3, 100);
           int rowsInserted = pstmt.executeUpdate();
           System.out.println(rowsInserted + " records are inserted");

           // partial changes
            pstmt.setString(1, "C++");
            rowsInserted = pstmt.executeUpdate();
            System.out.println(rowsInserted + " records are inserted");

            ResultSet rset = pstmtSelect.executeQuery();
            while (rset.next()) {
                String title = rset.getString("title");
                String price = rset.getString("price");
                int qty = rset.getInt("qty");
                System.out.println(title + ", " + price + ", " + qty);
            }

        }
    }
}
```

Atomic Transaction

```
package jdbcDemo;

import jdbcDemo.utils.JdbcConfig;

import java.sql.*;
```

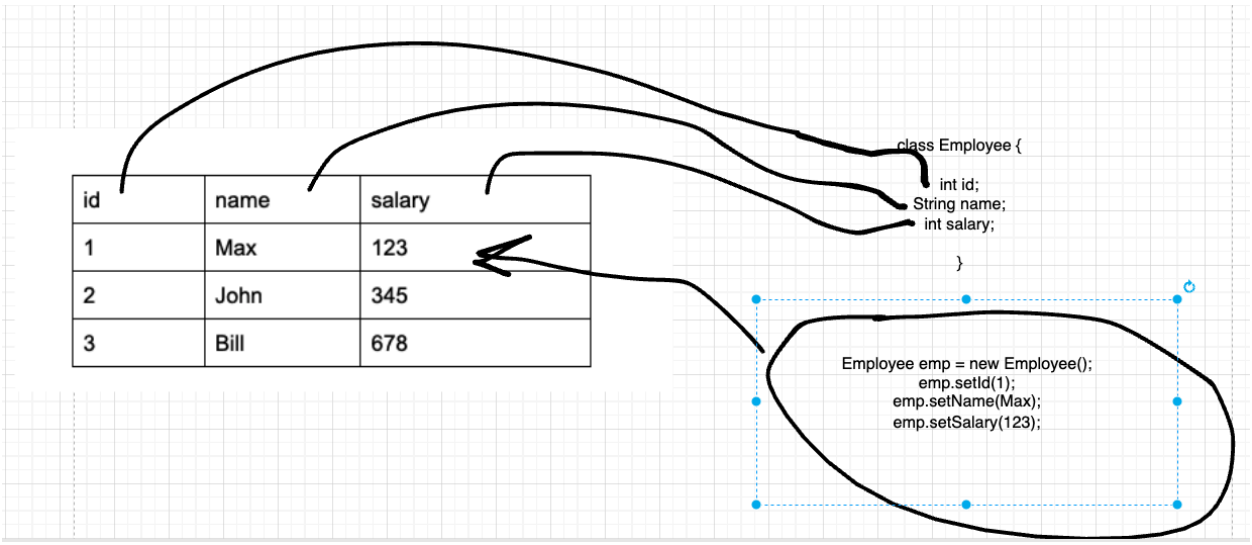
```
public class JdbcRollBackInCatchTest {
    public static void main(String[] args) throws SQLException {
        try (
                Connection conn = DriverManager.getConnection(
                        JdbcConfig.getUrl(),
                        JdbcConfig.getUser(),
                        JdbcConfig.getPassword()
                );
                Statement stmt = conn.createStatement();
        ) {
            try {
                conn.setAutoCommit(false);
                // insert two statements , title is the primary key, (AWS
duplicated keys, exception)
                stmt.executeUpdate("insert into books values ('AWS', 98, 12)");
                // duplicate primary key, which will trigger a SQLException
                stmt.executeUpdate("insert into books values ('AWS', 90, 15)");
                conn.commit();
            } catch (SQLException throwables) {
                System.out.println("rolling back changes");
                conn.rollback();
                throwables.printStackTrace();
            }


        }
    }
}
```

2. Hibernate

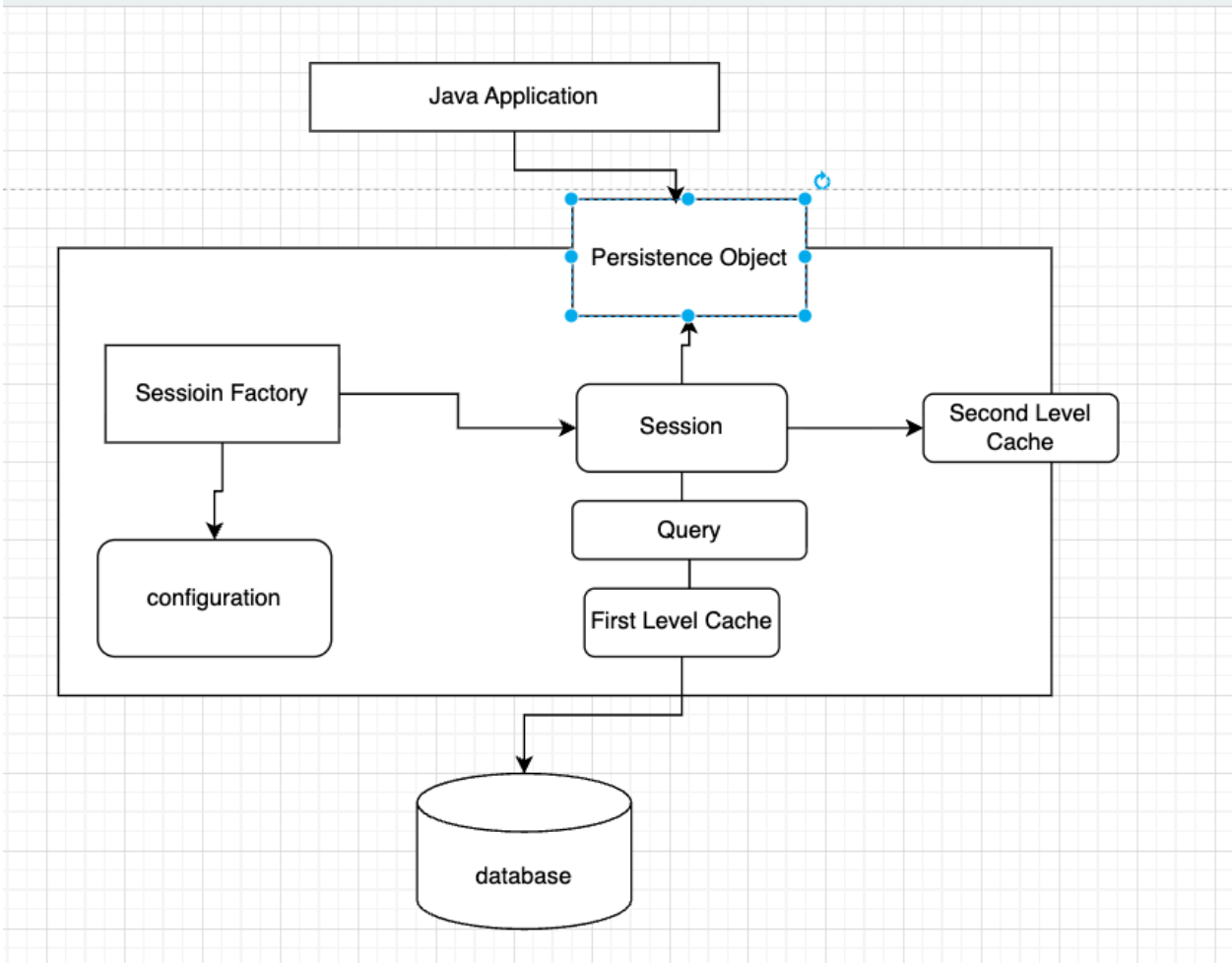
ORM (object relational mapping)

id	name	salary

1	Max	123
2	John	345
3	Bill	678



Hibernate Architecture
first level cache
second level cache



Mapping

- one to one
- one to many
- many to many

employeeId	email	firstName	lastName
1			
2			
3			

accountId	accountNumber	join column
A		1
B		1
C		2

emp1 accounts <A, B>
emp2 accounts <C>

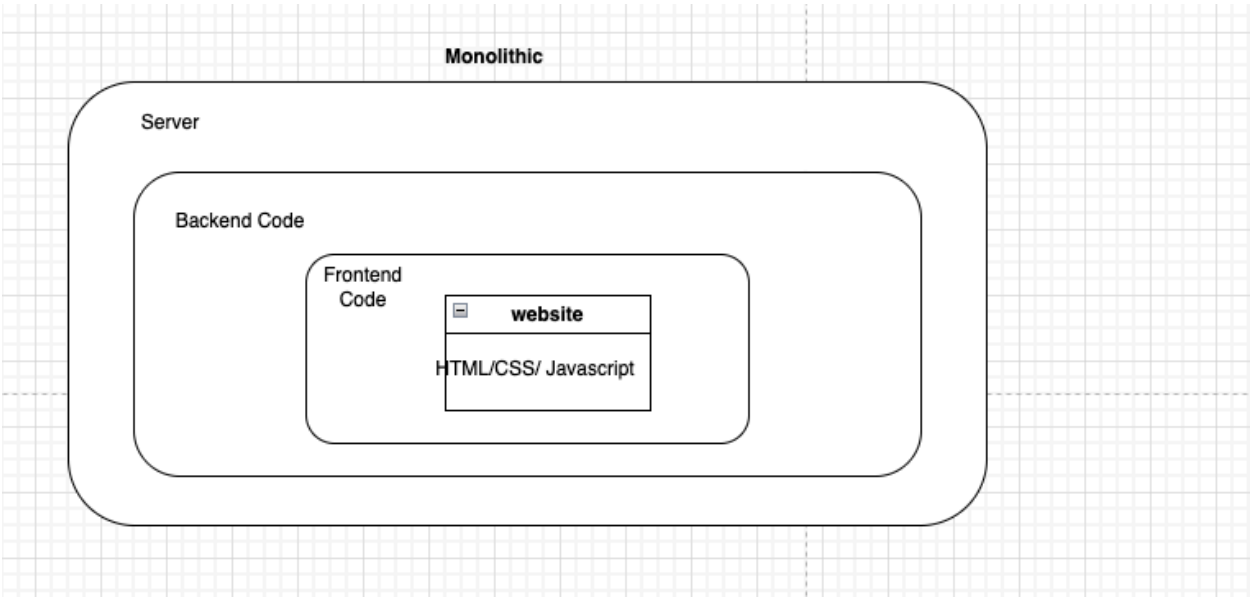
join table

id(primary key)	employeeID	accountID
	1	A
	1	B
	2	C

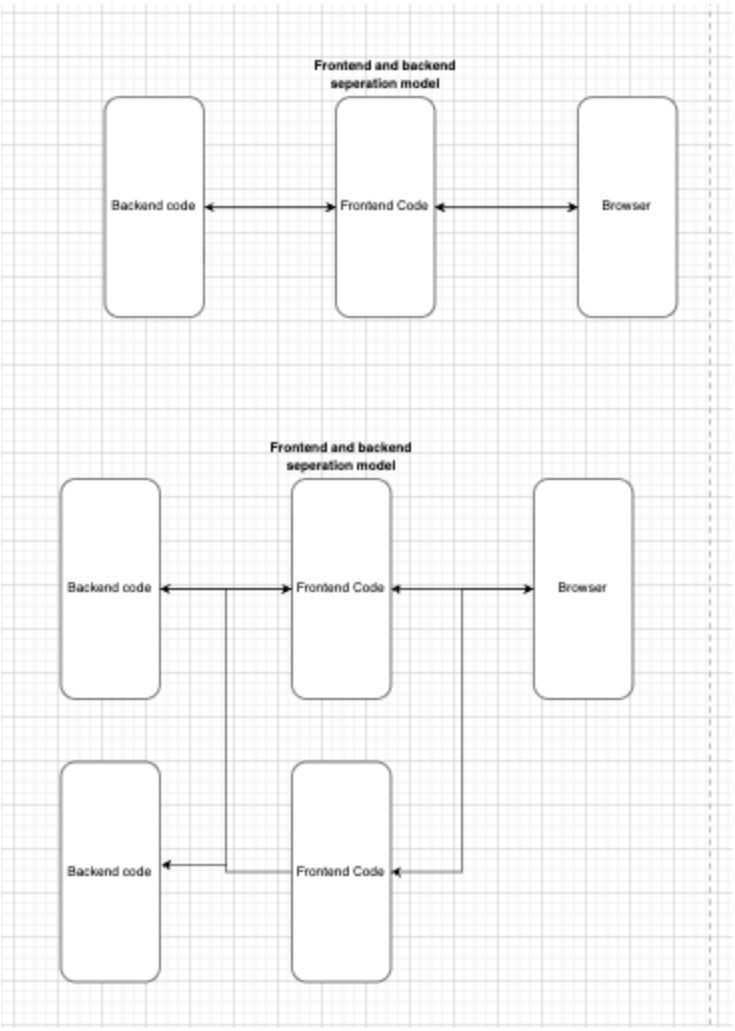
Advanced Web

1. Evolution of web architecture

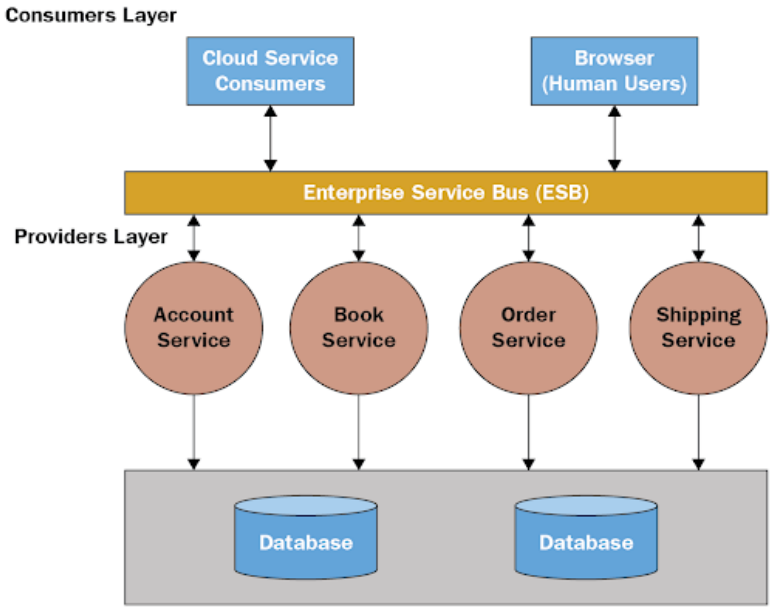
Monolithic



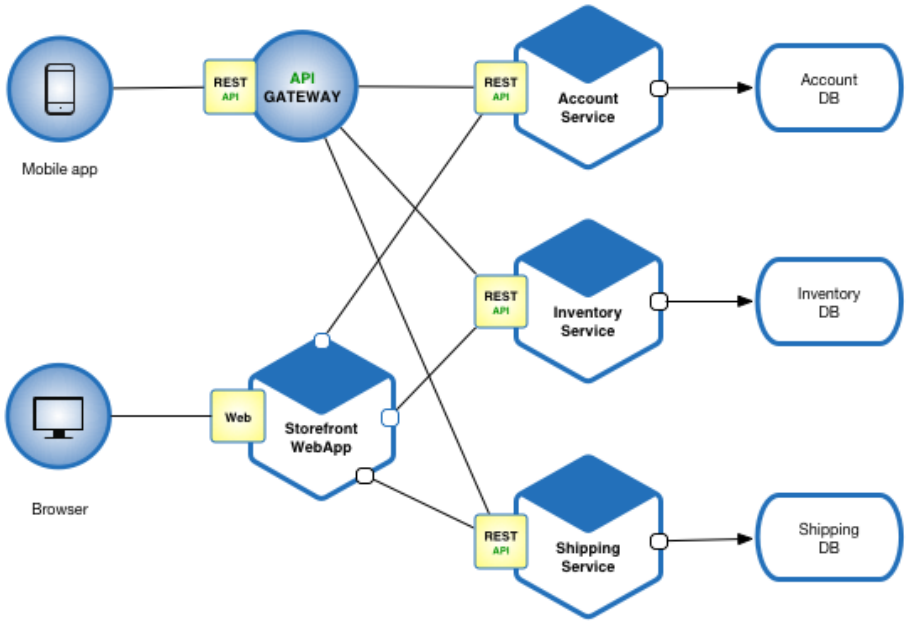
Frontend - backend separation


Frontend and backend seperation model
Backend code
Frontend Code
Browser
Frontend and backend seperation model
Backend code
Frontend Code
Browser
Backend code
Frontend Code

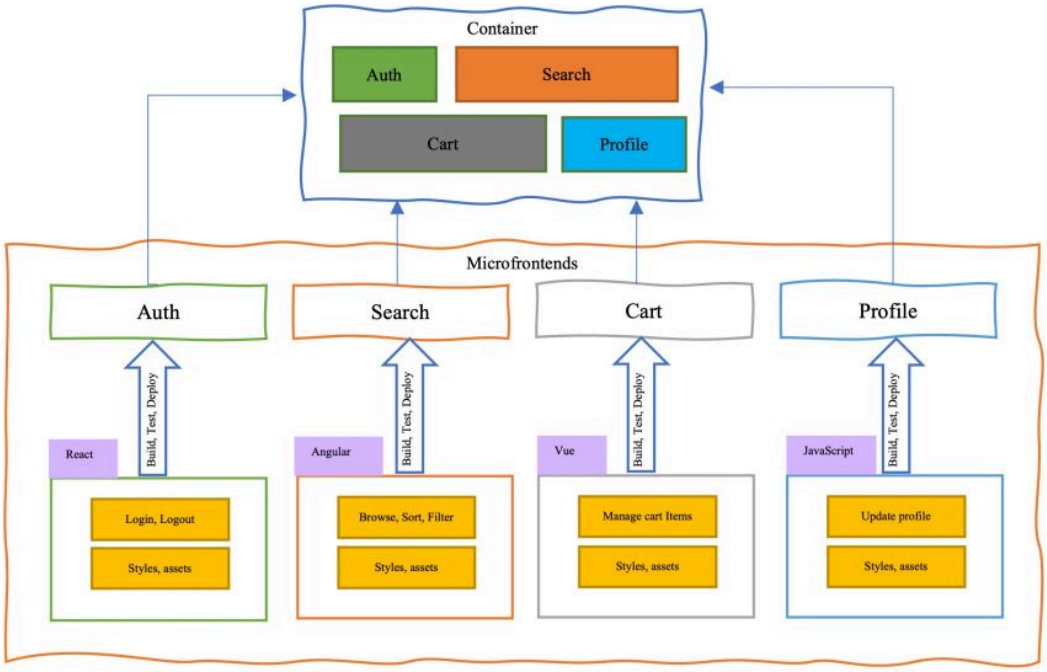
SOA -> ESB



Microservice

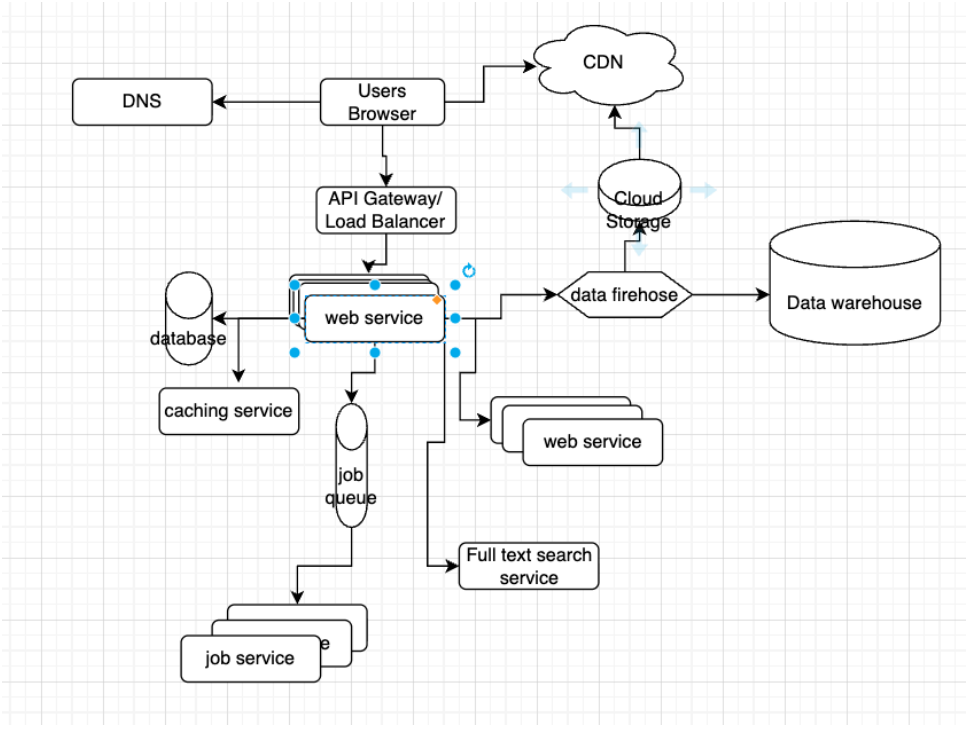


Micro-frontend framework
webpack



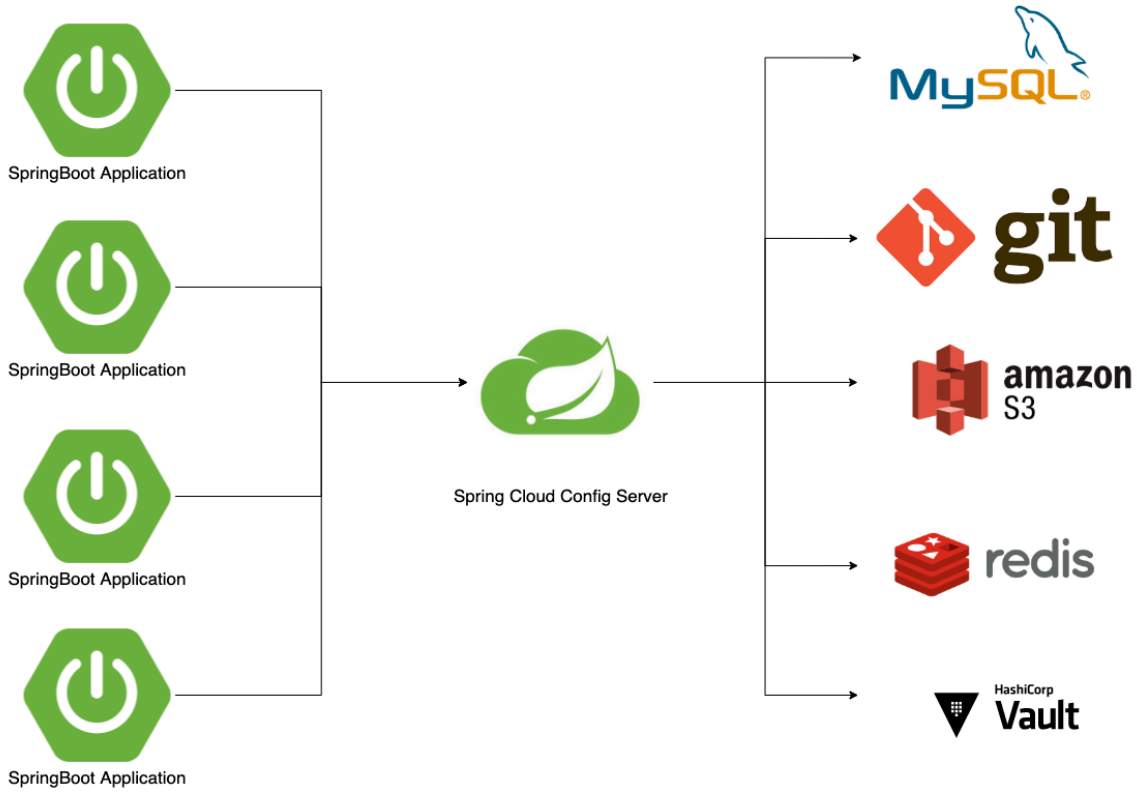
BFF (backend for frontend)

2. One typical web architecture example



3. Spring Cloud

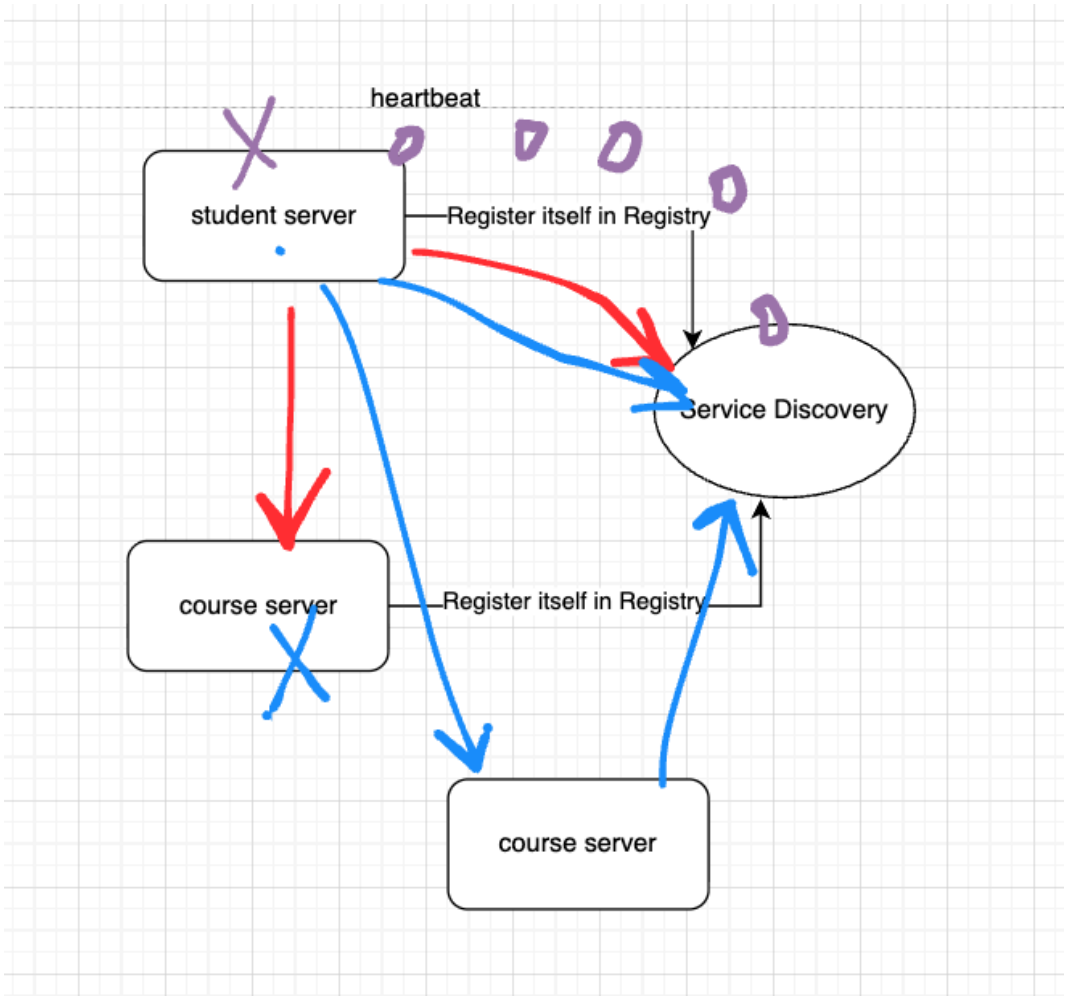
1. Config Server



2. Service Discovery and Registry

- Eureka

other options: zookeeper



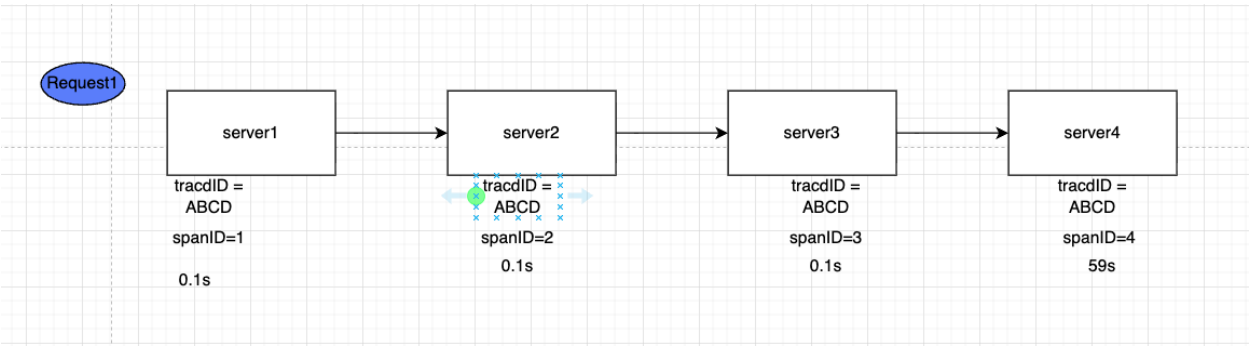
3. Zipkin and Slerth

Zipkin: distributed tracking system.

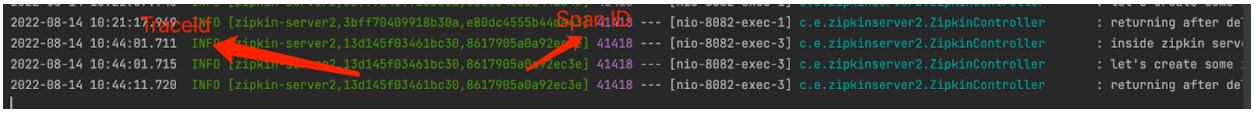
- collector
- storage
- search
- webUI

Sleuth:

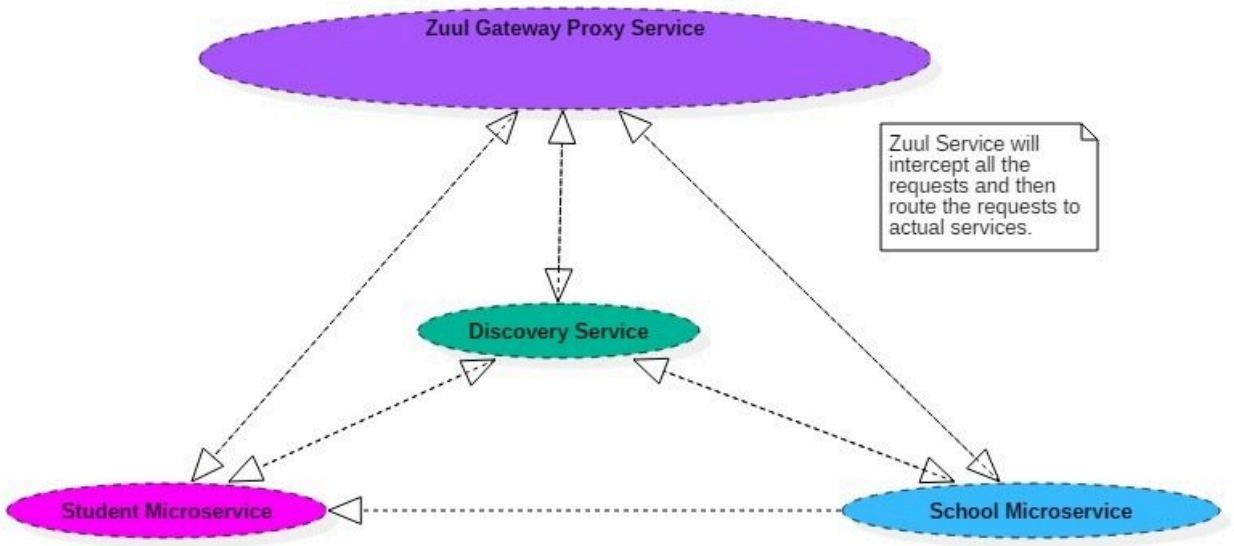
- traceID
- spanID

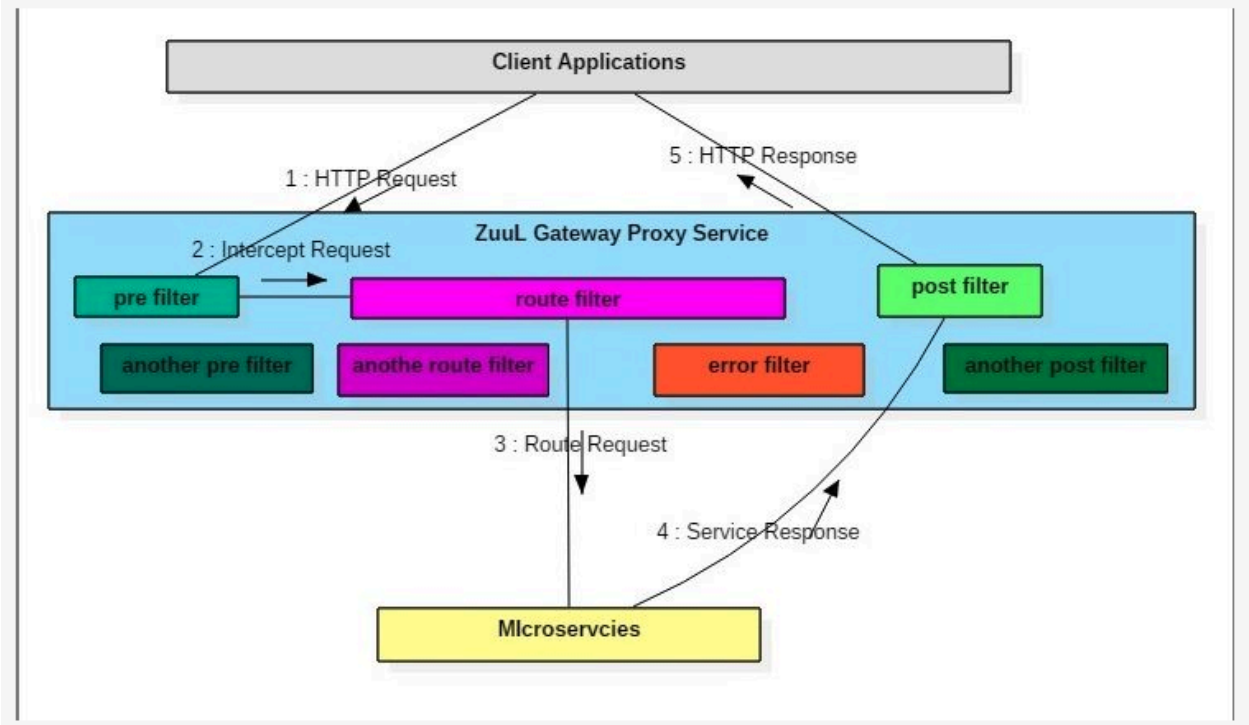


AWS X-ray



4. Zuul API Gateway





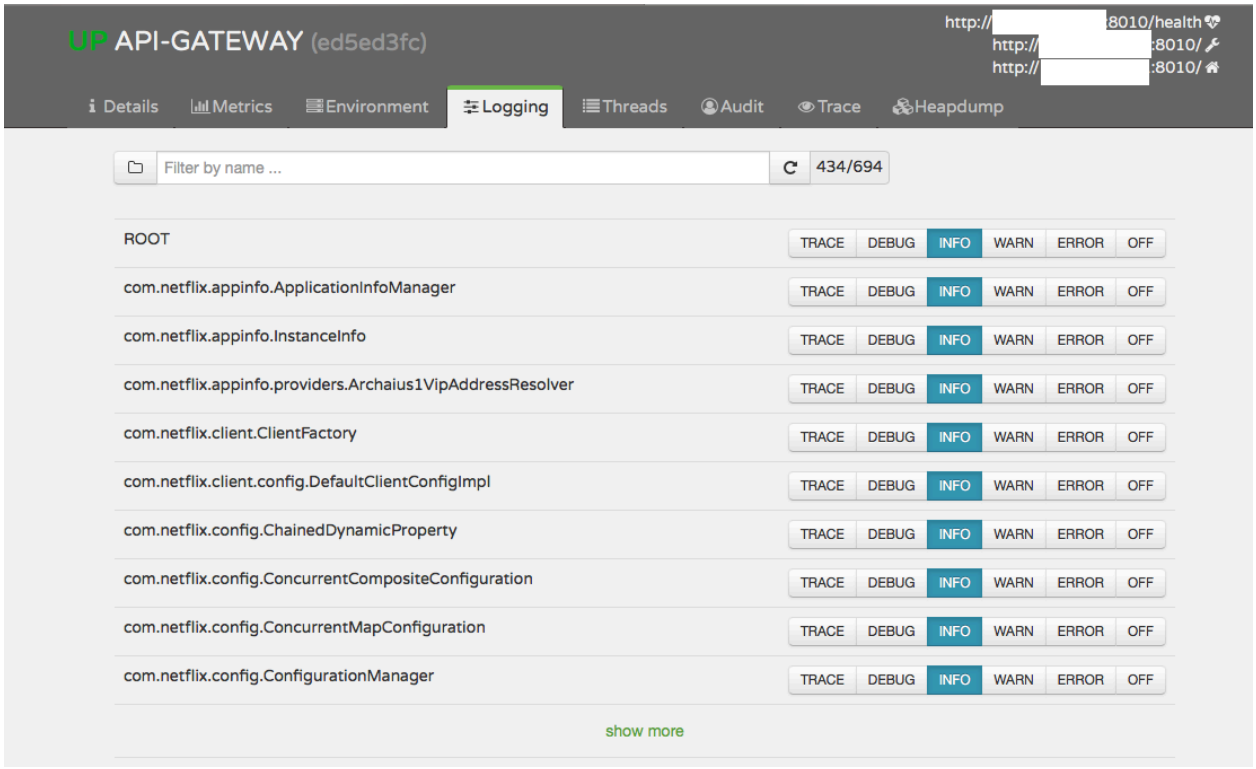
Request Processing flow inside Zuul with different filters

5. Ribbon

Load balancer

6. Spring boot Admin

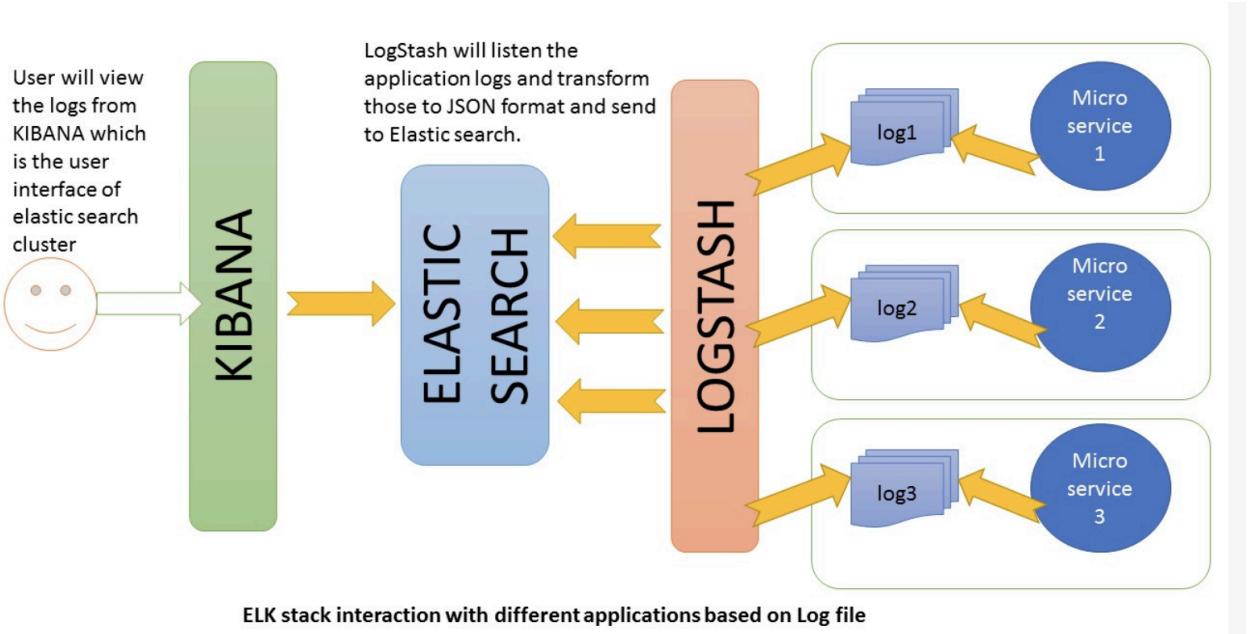
Service Monitor



Netflix Hystrix, Feign etc

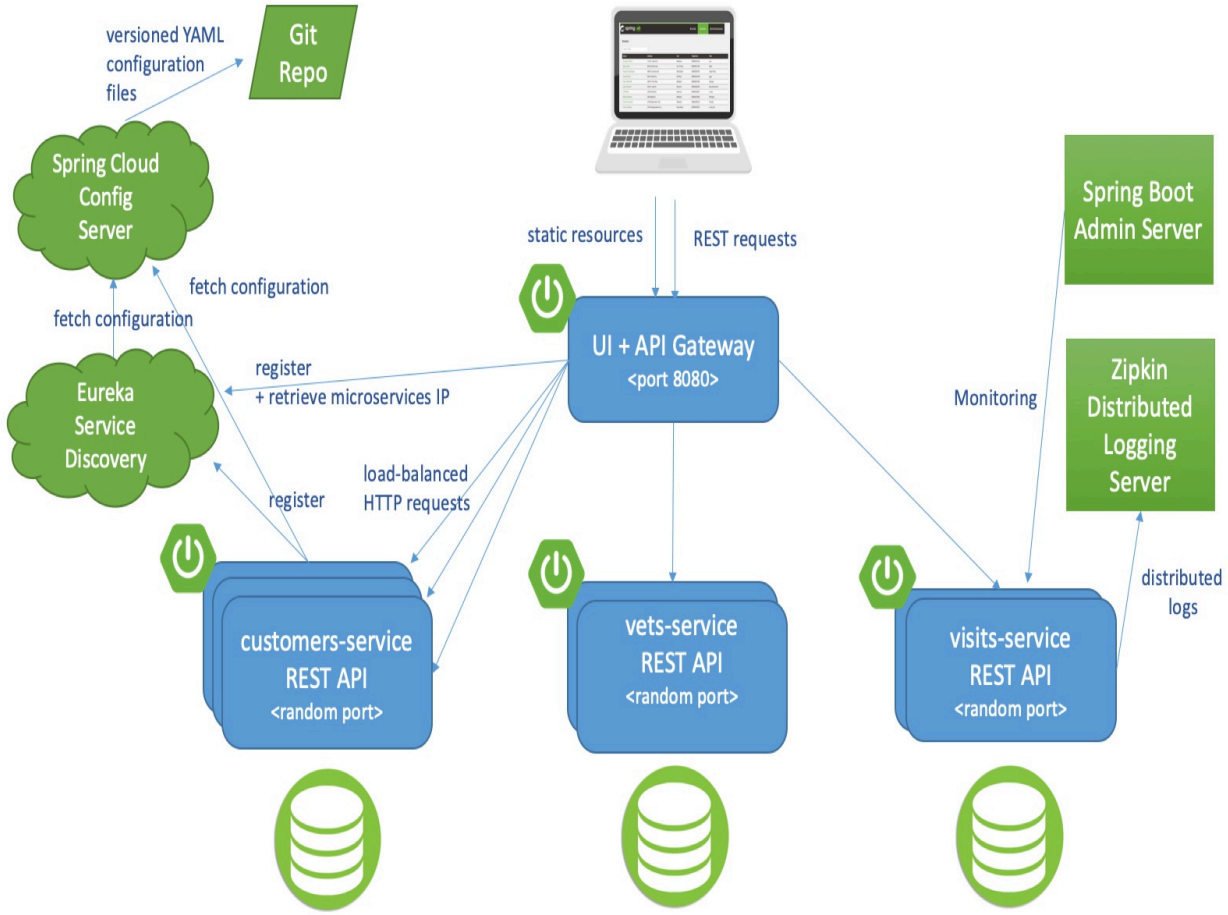
ELK - Other Service not belongs to Spring Cloud

Elasticsearch
Logstash
Kibana



ELK In Action

Spring Cloud Based Microservice Architecture



In-Class Project

#A retailer offers a rewards program to its customers, awarding points based on each recorded purchase. A customer receives 2 points for every dollar spent over $100 in each transaction, plus 1 point for every dollar spent over $50 in each transaction (e.g. a $120 purchase = 2x$20 + 1x$50 = 90 points). Given a record of every transaction during a three month period, calculate the reward points earned for each customer per month and total.

- The package name is structured as com.retailer.rewards
- An exception is thrown if a customer does not exist.
- H2 in-memory database to store the information.
- Install H2 db locally and run it . change the db settings in application.properties file.
- Do run the scrip.sql on H2 in memory DB to prepare the test data.

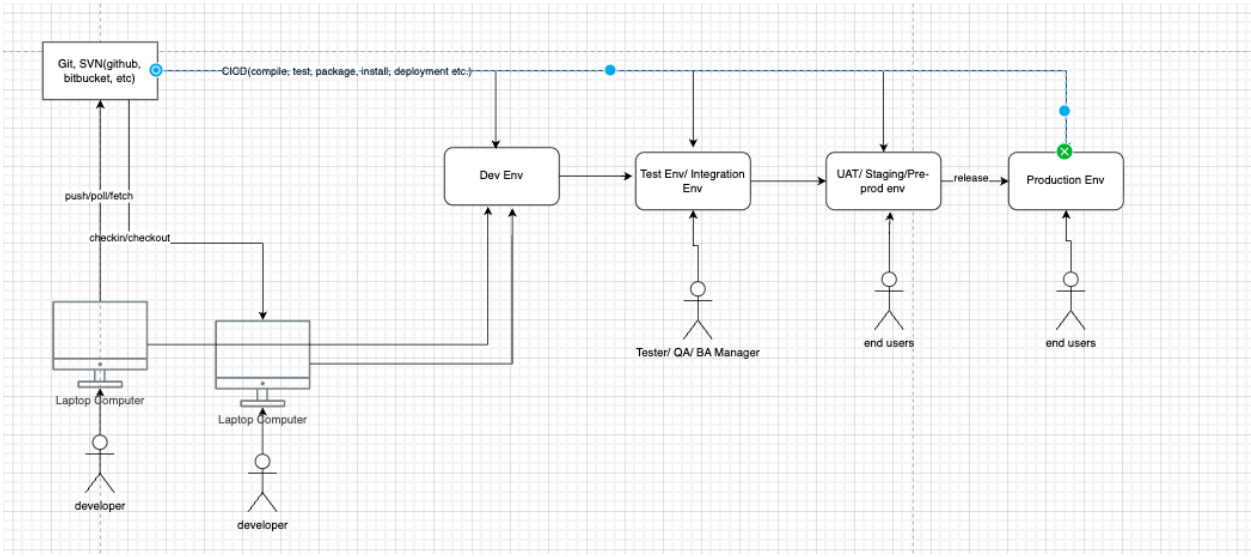
transaction:

1. data.sql file in resources
2. restful api for transaction

rewards API

Env, CICD,Git, Monitor

1. Environment



Dev Env

- developer
- very frequent
- unit/ integration test

Test Env/ Integration Env

- unit/ integration test, penetration test
- Tester, BA, Product Manager
- maybe be two weeks

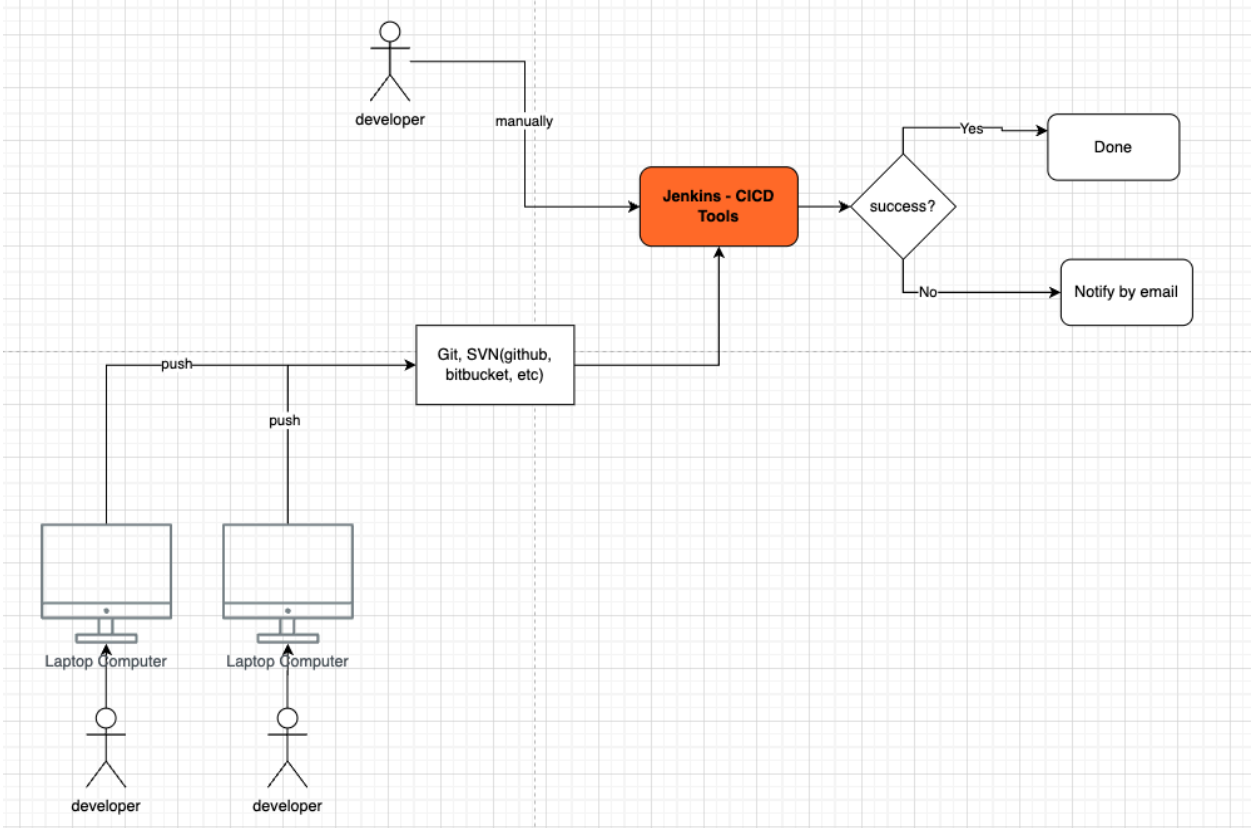
UAT (User Acceptance Test)/ Staging/ Pre-Prod

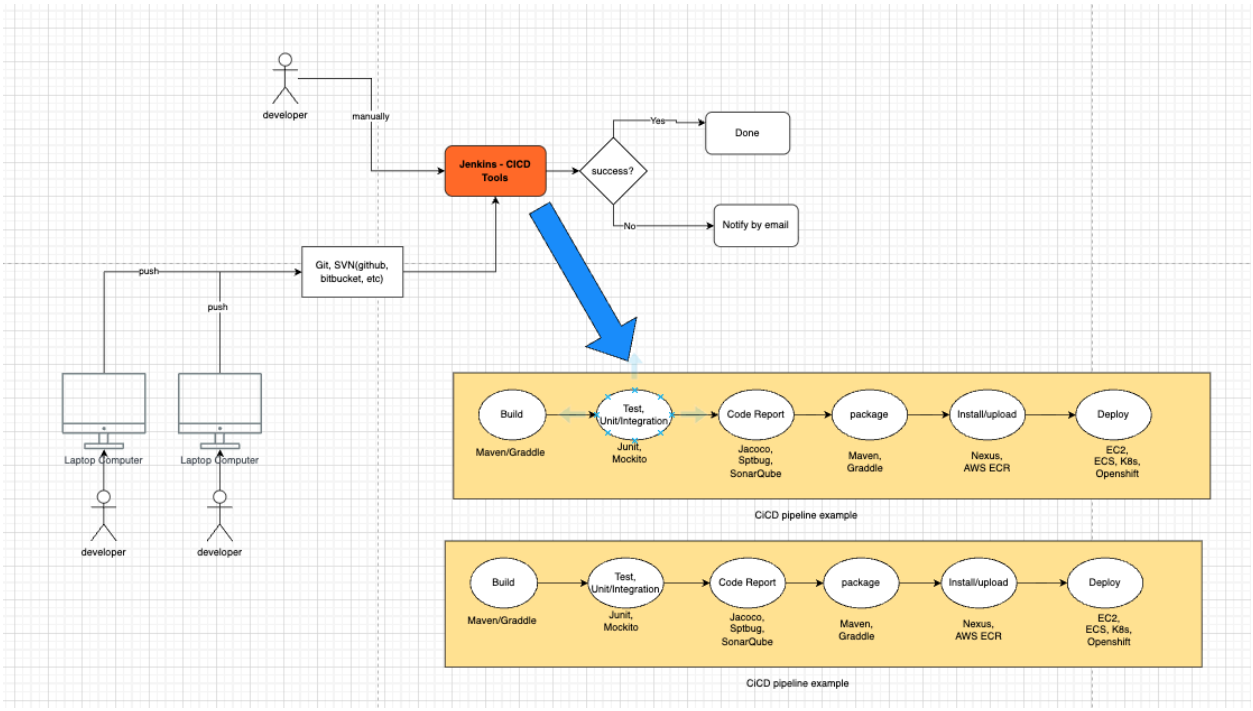
- Performance Test (JMeter)

- some other end users
- internal network
- deploy period monthly

Production Env
- release
- deploy period monthly

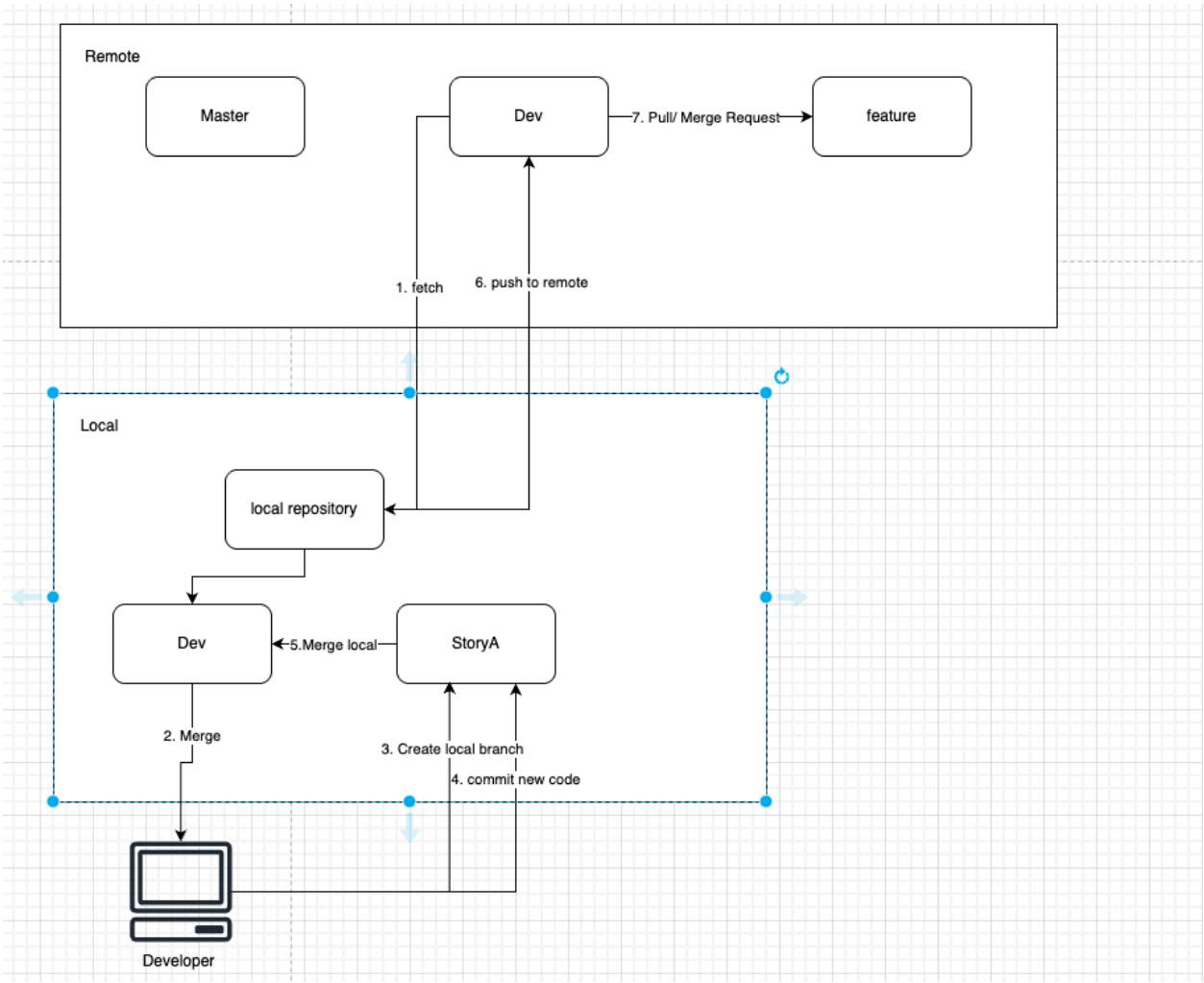
2. CICD





Jenkins: CICD Tools
Jenkinsfile,

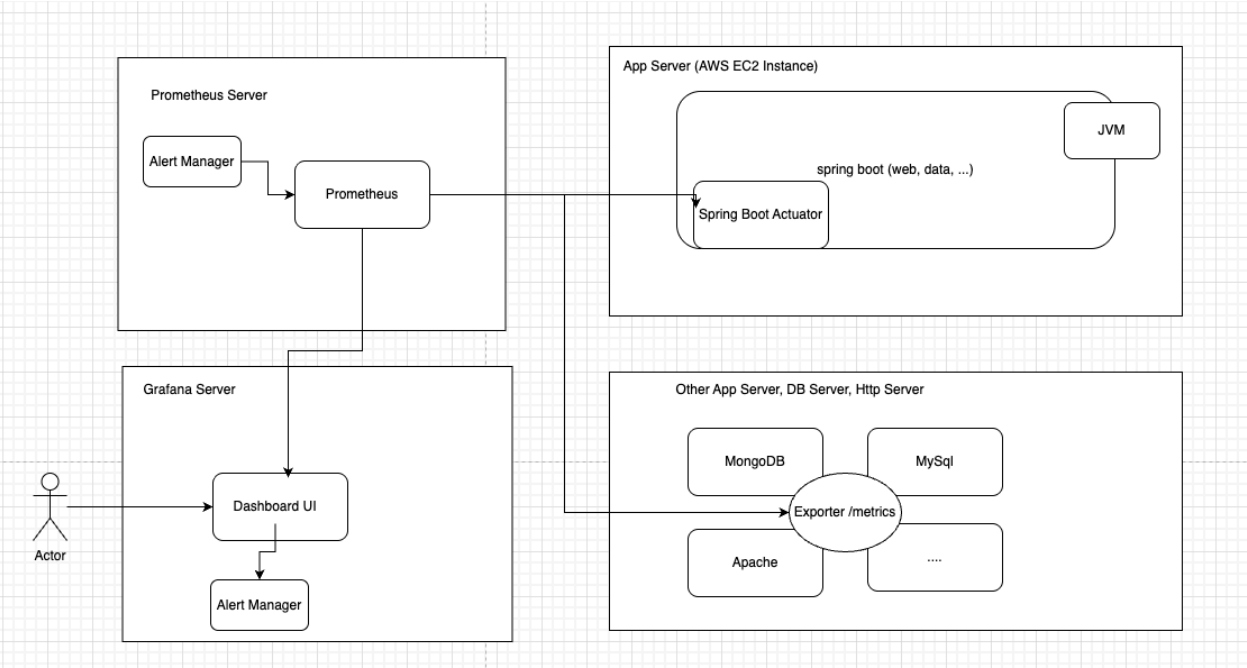
3. Git



Shell

```
git fetech origin

git checkout dev
git merge origin/dev

git check -b feature/storyA

git add .
git commit -m "implement story A"

git push
```

4. Monitor



prometheus/ Grafana
AWS CloudWatch

Agile Scrum

1. Title

Team title
BA(business analyst), QA(Tester), DBA(database admin), DevOps, Developer
TL, Architect, Product Manager

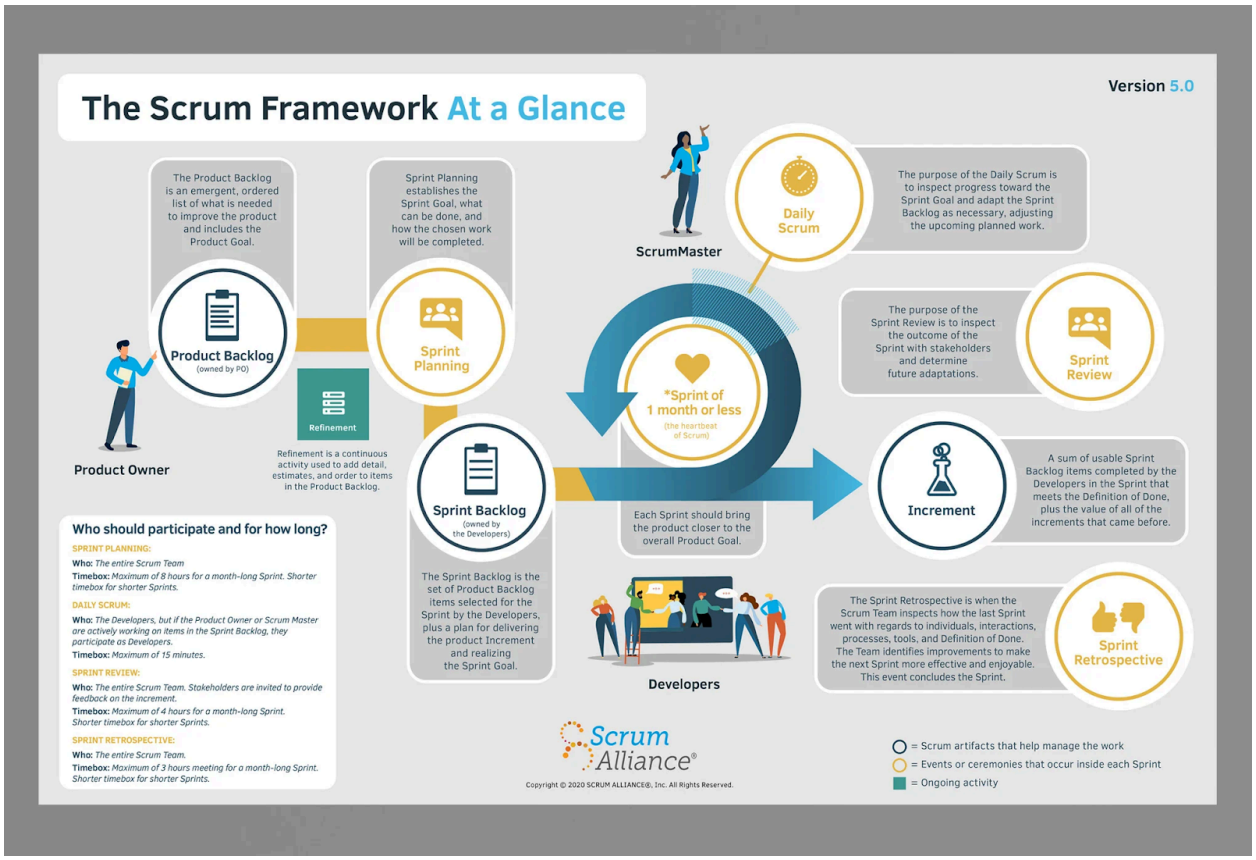
Agile title
Scrum Master: person who leads the process, meeting
Product Owner: owns the product

team size 6-10
for example: 6
1 tech lead, 1 QA, 1 BA, 3 Developer (full stack)

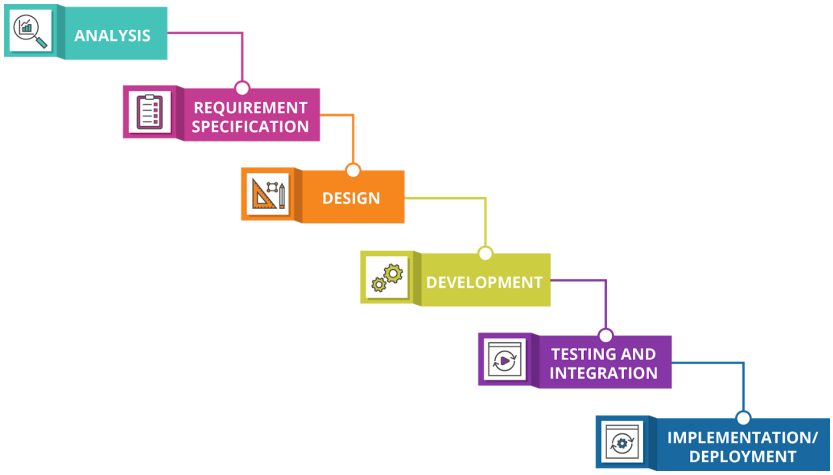
2. Meeting

Sprint Planning Meeting
Daily Scrum Meeting (Daily Stand-Up Meeting) (DSU)
Sprint Review Meeting
Sprint Retrospective Meeting

Sprint: max 1 month, normally 2 - 4 weeks



3. Agile vs Waterfall Methodology



4. Story Points

point: 1point = 2 hours
if one sprint is one week, 40 hours, normally give 16 points, 32 hours, margin

Fibonacci
1, 2, 3, 5, 8, 13….
13, too big -> 5, 8 -> 5, 5, 3

team size <= 10

5. If you meet Difficulties

if business difficulties, go for BA
if tech difficulties, go for TL
if there are difficulties related to other team, like API, contract stuff -> get confirmation from team lead -> reach out other team developer

6. Tools: Jira